

Índice general

I Creación de preguntas de opción múltiple	3
1. Clase Marcador	6
1.1. Funcionamiento General	6
1.2. Implementación	6
1.2.1. Método <code>__init__(self, data)</code>	7
1.2.2. Método <code>__iter__(self)</code>	7
1.2.3. Método <code>__next__</code>	7
2. Clases Supuesto y Cuestion	9
2.1. Clase <code>Supuesto(enunciado, semantica, precondition)</code>	9
2.2. Clase <code>Cuestion(enunciado, semantica, precondition)</code>	9
3. Segmentación en partes y validación de homogeneidad	11
3.1. Comprobación	12
4. Clase ProblemaTipo	13
4.1. Implementación	13
4.1.1. Método <code>__iter__(self)</code>	13
4.1.2. Método <code>__next__</code>	14
4.2. Ejemplo de funcionamiento	15
5. Clase ProblemaTipoProfe	19
5.1. Implementación	19
5.1.1. Método <code>__next__</code> para la clase <code>ProblemaTipoProfe</code>	19
5.1.2. Función auxiliar <code>CuestionesJuntas</code>	20
5.2. Ejemplo de funcionamiento	20
6. Clase ProblemaVF	22
6.1. Implementación	22
6.1.1. Métodos de la clase <code>ProblemaVF</code>	22
6.2. Ejemplo de funcionamiento	23
6.2.1. Generación de cuatro variantes cortas	23
6.2.2. Generación de cinco variantes con presentación formateada	23
7. Clase SubPregunta y ProblemaMultiParte	25
7.1. Ejemplo	26
II Serialización y persistencia en formato JSON	27
8. Formato JSON	30
8.1. Para un <code>ProblemaTipo</code>	30
8.2. Para un <code>ProblemaVF</code>	31
9. Evaluación de expresiones en el espacio de nombres de <code>calcpop</code>	32
10.Función de <code>setup</code> desde código Python	33

11.Deserialización de componentes	34
12.Serialización de componentes	35
13.Serialización de problemas completos	36
14.Lectura y escritura de ficheros	38
15.Ejemplo de uso	39
16.Exportación a código Python editable	40
16.1. <code>_slot_to_python</code>	40
16.2. <code>_subpregunta_to_python</code> , <code>problema_to_python</code> y <code>save_problema_py</code>	42
16.3. Ejemplo	42
 III Editor visual de problemas en Jupyter	 43
17.Dependencias e instalación	45
18.Widget de alternativa (<code>_AltWidget</code>)	47
19.Widget de slot (<code>_SlotWidget</code>)	49
20.Editor principal (<code>ProblemaTipoEditor</code>)	51
 IV ¿Y ahora qué?	 55
20.1. ¿Qué tenemos?	56
20.2. ¿Qué nos falta?	56

Parte I

Creación de preguntas de opción múltiple

Estructura del módulo qbank._quiz

El módulo completo está compuesto varios fragmentos de código se son descritos en los siguientes capítulos.

```
qbank/_quiz.py
from calccprop import *
from string import Template

<<Funciones auxiliares para setup paramétrico>>
<<Metodo auxiliar para juntar cuestiones en formato para profe>>
<<Definición de la clase Marcador>>
<<Definición de la clase Supuesto>>
<<Definición de la clase Cuestion>>
<<Segmentación en partes y validación de homogeneidad>>
<<Definición de la clase ProblemaTipo>>
<<Definición de la clase ProblemaTipoProfe>>
<<Definición de la clase ProblemaVF>>
<<Definición de SubPregunta y ProblemaMultiParte>>

<<Interfaz gráfica para el diseño visual de la lista de listas de ejercicios>>
```

Uso del módulo

Para usar este módulo, primero se debe ejecutar

```
from qbank import *
```

El fichero `__init__.py` del paquete reúne la interfaz pública reexportando los submódulos. El editor visual (`_widgets`) se importa dentro de un `try/except` porque depende de `ipywidgets`, que es una dependencia opcional: si no está instalada, el resto del paquete sigue funcionando.

```
qbank/__init__.py
from qbank._quiz import *
from qbank._export import *
from qbank._json import *
try:
    from qbank._widgets import *
except ImportError:
    pass
```

Funciones auxiliares para setup paramétrico

Las funciones `_ns_eval` y `_ns_interp` permiten que los atributos de `Supuesto` y `Cuestion` (enunciado, semántica, precondition) sean valores directos o **callable**s que reciben el namespace generado por el setup de `ProblemaTipo`.

`_ns_eval(val, ns)` si `val` es callable, lo invoca con `ns` y devuelve el resultado; si no, devuelve `val` tal cual.

`_ns_interp(s, ns)` interpola en la cadena `s` los marcadores `@variable` usando el diccionario `ns`. Usa `string.Template.safe_substitute` con delimitador `@` (en lugar del `\$` estándar, para evitar conflictos con el modo matemático de $\text{\LaTeX}$). Los marcadores desconocidos y las llaves `{...}` de $\text{\LaTeX}$ no se ven afectados. Para obtener una arroba literal en el texto, escribe `@@`.

`_fuente_precond(componente)` devuelve una representación **legible** de la precondition de un `Supuesto` o `Cuestion` para los mensajes de rechazo. Si el componente se cargó desde JSON (atributo `_json`) usa la cadena de código original (p. ej. `lambda ns: ns['a'] > ns['b']`); en otro caso recurre a `repr()` (que para una lambda definida en Python da algo como `<function <lambda> at 0x...>`). Así los avisos «=... rechazado por ...=» muestran la condición y no la dirección de memoria de la función.

Funciones auxiliares para setup paramétrico

```
class _NsTemplate(Template):
    delimiter = '@'

def _ns_eval(val, ns):
    return val(ns) if callable(val) else val

def _ns_interp(s, ns):
    return _NsTemplate(s).safe_substitute(ns) if ns else s

def _fuente_precond(componente):
    """Representación legible de la precondition de un componente.

    Si el componente se cargó desde JSON, devuelve la cadena de código
    original (p. ej. ``lambda ns: ns['a'] > ns['b']``); en otro caso usa
    ``repr()`` , que para una lambda definida directamente en Python da algo
    como ``<function <lambda> at 0x...>``.
    """
    j = getattr(componente, '_json', None)
    if j and j.get('precond') is not None:
        return j['precond']
    return repr(componente.p)
```

Capítulo 1

Clase Marcador

Usaremos la clase `Marcador` como una herramienta auxiliar para construir el árbol con todas las combinaciones posibles entre un conjunto de proposiciones y textos (que componen un conjunto de posibles enunciados), y un conjunto de cuestiones relacionadas con dichos enunciados.

(Posteriormente se descartarán de dicho árbol las combinaciones enunciado-cuestión en las que la veracidad o falsedad de la cuestión no se pueda deducir con las proposiciones declaradas en el enunciado.)

1.1. Funcionamiento General

Invocamos esta clase del siguiente modo: `Marcador(data)`, donde `data` es una lista de números enteros mayores que cero. Por ejemplo, si invocamos `Marcador([2, 3, 1])`, estamos pidiendo todas las listas `[i, j, k]` tales que $0 \leq i < 2$, $0 \leq j < 3$ y $0 \leq k < 1$ (es decir, la primera componente solo puede tomar dos valores (0 y 1), la segunda tres (0, 1 y 2) y la tercera solo puede valer 0).

```
for i in Marcador([2,3,1]):  
    print(i)
```

```
[0, 0, 0]  
[0, 1, 0]  
[0, 2, 0]  
[1, 0, 0]  
[1, 1, 0]  
[1, 2, 0]
```

1.2. Implementación

La clase `Marcador` es un [iterador](#). Para el argumento `data=[n_1,...,n_k]` genera todas las listas de la forma `[m_1,...,m_k]` tales que cada `m_i` es un número natural menor que `n_i`.

Esta clase `Marcador` está formada por los siguientes fragmentos de código:

```
class Marcador:  
    <<Método __init__ para la clase Marcador>>  
    <<Método __iter__ para la clase Marcador>>  
    <<Método __next__ para la clase Marcador>>
```

`__init__` El método `__init__` es el inicializador de instancias en Python y actúa como el constructor de la clase; su función principal es configurar el estado inicial de un objeto recién creado asignándole valores a sus atributos mediante el argumento `self`. Este método especial se ejecuta de forma automática y obligatoria cada vez que se instancia una clase, permitiendo que cada objeto nazca con datos propios y personalizados, lo que evita tener que definir las variables de manera manual tras la creación del objeto. Para profundizar en los detalles técnicos de su funcionamiento, la herencia y las buenas prácticas, puede consultar la sección sobre clases en la [Documentación Oficial de Python](#).

Además, para construir un iterador necesitamos los métodos `__iter__` y `__next__`.

__iter__ El método especial `__iter__` es el encargado de hacer que un objeto sea iterable en Python, permitiendo que pueda ser utilizado directamente en bucles `for`, comprensiones de listas o cualquier contexto que requiera recorrer elementos uno a uno. Su papel preciso es devolver un objeto iterador (que puede ser el propio objeto `self` si este implementa el método `__next__`, o un objeto iterador dedicado de otra clase), estableciendo así el protocolo de iteración de Python. Al invocar este método, Python sabe exactamente cómo comenzar el recorrido de la estructura de datos, ya sea una colección nativa como una lista o una clase personalizada que hayas diseñado. Para comprender a fondo cómo implementar este método junto con el protocolo de iteración y los generadores, puede consultar la sección sobre iteradores en la [Documentación Oficial de Python](#).

__next__ El método especial `__next__` es la pieza fundamental que hace funcionar a un objeto **iterador** en Python, siendo el encargado de devolver el siguiente elemento disponible cada vez que se avanza en una secuencia. Este método es invocado automáticamente entre bastidores por la función nativa `next()` o al recorrer un bucle, y tiene la responsabilidad crucial de lanzar la excepción estándar `StopIteration` cuando se han agotado todos los elementos, lo que le indica a Python que la iteración ha terminado de forma segura. Junto con `__iter__`, conforma el protocolo de iteración completo de Python, permitiendo un control milimétrico sobre cómo se calculan y entregan los datos (incluso bajo demanda o de forma infinita). Para ver ejemplos prácticos de su implementación y entender cómo gestiona el flujo de datos, puede revisar la documentación sobre tipos iteradores en la [Referencia de la Biblioteca Estándar de Python](#).

1.2.1. Método `__init__(self, data)`

Este método inicializa la clase y almacena en el atributo `self.data` el argumento `data`; que es una lista con los topes de cada una de las componentes de las listas que se van a crear. En el ejemplo anterior, `[2,3,1]`.

```
def __init__(self, data):  
    self.data = data
```

1.2.2. Método `__iter__(self)`

El método `__iter__` inicializa el estado del iterador. Este método genera la semilla del iterador. Dicha semilla es una lista de ceros (con tantos ceros como elementos hay en `self.data`). La lista `self.p` se entregará a `__next__` en la siguiente invocación al método.

```
def __iter__(self):  
    self.p = [0 for x in self.data]  
    return self
```

1.2.3. Método `__next__`

Gestiona el avance del iterador.

- `__next__` devuelve `self.p` en su estado actual (`n`), y coloca en `self.p` la lista siguiente usando la función auxiliar **Siguiente**.
- Si `self.p` es vacía detiene la iteración.

```
def __next__(self):  
    <<Definición de la función local Siguiente>>  
    if self.p == []:  
        raise StopIteration  
    n = self.p  
    self.p = Siguiente(self.p, self.data)  
    return n
```

1. Función auxiliar **Siguiente** (`x, y`)

Dentro del método `__next__` definimos de manera recursiva la función auxiliar local **Siguiente**, que es la función que calcula la lista siguiente.

La función **Siguiente**(**x**, **y**) es el corazón lógico de la clase. Utiliza **recursividad** para implementar un orden lexicográfico:

- **x**: lista de números naturales [**m**₁...**m**_r] tales que **m**_i < **n**_i (es la lista de la que queremos calcular la siguiente).
- **y**: lista de números naturales [**n**₁...**n**_r] mayores que cero (lista de topes).

La función **Siguiente** devuelve una lista [**k**₁...**k**_r]; que es la siguiente lista a [**m**₁...**m**_r] (según el orden lexicográfico) que aún cumple que **k**_i < **n**_i. En caso de no existir tal lista devuelve [].

a) Implementación:

- Intenta incrementar primero el último elemento de la lista (el de la derecha).
- Si el elemento de la derecha alcanza su límite (**x**[0]+1 == **y**[0]), intenta incrementar el elemento anterior y resetea.^a cero todos los elementos a su derecha.
- Si todos los elementos han alcanzado su límite, devuelve una lista vacía [], lo que señala el fin de la iteración.

Definición de la función local Siguiente

```
def Siguiente(x,y):
    if x == [] :
        return []
    s = Siguiente(x[1:],y[1:])
    if s == []:
        if x[0]+1 == y[0]:
            return []
        else:
            return [x[0]+1] + [0 for i in x[1:]]
    else:
        return [x[0]] + s
```

Capítulo 2

Clases Supuesto y Cuestion

2.1. Clase Supuesto(enunciado, semantica, precondition)

La clase `Supuesto` define la estructura para almacenar tres aspectos (atributos) del planteamiento de un problema mediante su constructor `__init__`.

enunciado cadena de texto que presenta los supuestos en lenguaje humano.

semantica proposición que traduce parcialmente el significado lógico del **enunciado** (contiene lo necesario para deducir las respuestas).

precond es una proposición o bien cualquiera de los valores `True` o `False`. El objeto `Supuesto` será incluido en el problema solo si la evaluación de **precond** es `True`.

Definición de la clase Supuesto

```
class Supuesto:
    def __init__(self, enunciado, semantica, precondition=True):
        self.e = enunciado
        self.s = semantica
        self.p = precondition

    def __repr__(self):
        """Método de representación"""
        return f"Supuesto(enunciado={self.e!r}, semantica={self.s!r}, precondition={self.p!r})"
```

2.2. Clase Cuestion(enunciado, semantica, precondition)

De manera análoga, la clase `Cuestion` define la estructura para almacenar cuatro aspectos (atributos) del planteamiento de un problema mediante su constructor `__init__`.

enunciado string que presenta la pregunta en lenguaje humano.

semantica proposición P tal que `test(P, semantica_de_los_supuestos)` devuelve `True` si no es refutable a partir de las premisas, es decir, es imposible que P sea falso si las premisas son verdad.

precond es una proposición o bien cualquiera de los valores `True` o `False`. El objeto `Cuestion` será incluido en el problema solo si la evaluación de **precond** es `True`.

exp al exportar las preguntas al formato `xml` de Moodle, cabe la posibilidad de que Moodle ofrezca al alumno una explicación una vez ha contestado a las preguntas. En el atributo **exp** se especifica la cadena de texto con la explicación que debe mostrar Moodle.

Definición de la clase Cuestion

```
class Cuestion:
    def __init__(self, enunciado, semantica, precondition=True, exp=""):
        self.e = enunciado
        self.s = semantica
        self.p = precondition
        self.x = exp

    def __repr__(self):
        """Método de representación"""
```

```
return (f"Cuestion(enunciado={self.e!r}, semantica={self.s!r}, "  
        f"precond={self.p!r}, exp={self.x!r})")
```

Capítulo 3

Segmentación en partes y validación de homogeneidad

La lista de componentes de un `ProblemaTipo` codifica implícitamente la estructura de un ejercicio (posiblemente multiparte) mediante dos invariantes:

- **(I1) Homogeneidad:** cada sublista contiene un único tipo de elemento (solo cadenas, solo `Supuesto` o solo `Cuestion`). Los escalares sueltos se interpretan como una sublista de un único elemento.
- **(I2) Segmentación:** un slot es *material de enunciado* (cadena, sublista de cadenas o sublista de `Supuesto`) o *cuestiones* (sublista de `Cuestion`). Una **parte** del ejercicio es una tanda de material de enunciado seguida de una tanda de cuestiones. Cuando, tras una tanda de cuestiones, reaparece material de enunciado, comienza una **parte nueva**. Un ejercicio de una sola parte es el caso degenerado.

Estas dos funciones implementan dichas reglas sin alterar todavía el comportamiento de `ProblemaTipo` (se usarán en fases posteriores):

- `validar_componentes(componentes)` comprueba I1 y lanza `ValueError` indicando el slot infractor.
- `segmentar(componentes)` valida y devuelve la lista de partes; cada parte es una tupla (`slots_enunciado`, `slots_cuestiones`).

Segmentación en partes y validación de homogeneidad

```
def _normaliza_slot(x):
    """Envuelve los escalares en una sublista; deja las listas tal cual."""
    return x if isinstance(x, list) else [x]

def _tipo_componente(e):
    if isinstance(e, str): return 'str'
    if isinstance(e, Supuesto): return 'Supuesto'
    if isinstance(e, Cuestion): return 'Cuestion'
    raise ValueError(
        f"Componente de tipo no soportado: {type(e).__name__} ({e!r}). "
        "Cada componente debe ser str, Supuesto o Cuestion.")

def _clasifica_slot(slot):
    """'cuestiones' si la sublista (ya normalizada y homogénea) es de Cuestion;
    'enunciado' en otro caso (cadenas o Supuesto)."""
    return 'cuestiones' if isinstance(slot[0], Cuestion) else 'enunciado'

def validar_componentes(componentes):
    """Comprueba la invariante de homogeneidad (I1).

    Tras normalizar, cada slot debe contener un único tipo entre
    {str, Supuesto, Cuestion}. Lanza ValueError indicando el slot infractor.
    """
    for i, x in enumerate(componentes):
        slot = _normaliza_slot(x)
        if not slot:
            raise ValueError(f"El slot {i} está vacío.")
        tipos = {_tipo_componente(e) for e in slot}
        if len(tipos) > 1:
            raise ValueError(
                f"El slot {i} mezcla tipos {sorted(tipos)}; cada sublista debe "
```

```

        "contener un único tipo (solo cadenas, solo Supuesto o solo Cuestion).")

def segmentar(componentes):
    """Divide la lista de componentes en partes según las invariantes I1 e I2.

    Devuelve una lista de tuplas (slots_enunciado, [slots_cuestiones]).
    Una transición cuestiones->enunciado abre una parte nueva. Un ejercicio
    de una sola parte produce una lista de longitud 1.
    """
    validar_componentes(componentes)
    L = [_normaliza_slot(x) for x in componentes]
    partes, enun, cues, vistas = [], [], [], False
    for slot in L:
        if _clasifica_slot(slot) == 'cuestiones':
            cues.append(slot)
            vistas = True
        else:
            if vistas:
                # I2: cierra parte y abre una nueva
                partes.append((enun, cues))
                enun, cues, vistas = [], [], False
            enun.append(slot)
    partes.append((enun, cues))
    return partes

```

3.1. Comprobación

Verificamos la segmentación (I2) y la validación de homogeneidad (I1):

```

S = lambda e: Supuesto(e, 'True')
C = lambda e: Cuestion(e, 'True')

# Una sola parte: texto + supuesto + un bloque de cuestiones.
assert len(segmentar(['Sea A.', S('A es par'), [C('q1'), C('q2')]])) == 1

# Dos partes: la transición cuestiones -> enunciado abre parte nueva.
assert len(segmentar(['Stem', [C('p1')], '(parte 2)', [C('p2')]])) == 2

# Tres partes con escalares y sublistas como slots distintos.
assert len(segmentar(['t0', [C('a')], 't1', S('h'), [C('b')], 't2', [C('d')]])) == 3

# Puede arrancar directamente con cuestiones (enunciado vacío en la 1ª parte).
assert segmentar([[C('a')], 'txt', [C('b')]])[0][0] == []

# Sublistas de cuestiones consecutivas pertenecen a la misma parte.
assert len(segmentar(['t', [C('a')], [C('b')], [C('c')]])[0][1]) == 3

# I1: una sublista que mezcla tipos se rechaza.
for bad in [[S('h'), C('q')], ['txt', C('q')], [[S('a'), 'c']], ['t', []], [42]]:
    try:
        validar_componentes(bad); raise AssertionError(f"no rechazó {bad!r}")
    except ValueError:
        pass

print("Fase 1 OK: segmentar + validar_componentes")

```

Capítulo 4

Clase ProblemaTipo

La clase `ProblemaTipo` permite, a partir de una lista de posibles enunciados y varias listas de posibles preguntas, generar muchas variantes de ejercicios de opción múltiple por simple combinatoria. Combina enunciados con una sublista de preguntas y, para cada pregunta calcula su veracidad o falsedad dados los supuestos del enunciado. Si no es posible decidir la veracidad o falsedad de una pregunta para un enunciado concreto, esa combinación enunciado-pregunta es descartada.

4.1. Implementación

La clase `ProblemaTipo` es un iterador que genera todas las versiones aceptables de un problema. Cada variante resulta de tomar una de las opciones de cada una de las listas en `supuestos_y_cuestiones`. Si la precondition de alguna de las opciones elegidas es `False` (o resulta refutable), la variante entera del problema es rechazada y se evalúa la siguiente combinación disponible.

- El atributo `self.e` contiene la lista `supuestos_y_cuestiones`. El argumento `supuestos_y_cuestiones` es una lista de listas de strings, objetos `Supuesto` o objetos `Cuestion`. Cada lista representa un conjunto de opciones excluyentes.

Por tanto, cada elemento de `supuestos_y_cuestiones` es a su vez una lista de opciones (excluyentes) correspondiente a cada componente (cada parte) del texto del problema.

Por comodidad, en el caso de que una de esas listas consista en un único objeto (una única opción), también se permite escribir directamente dicho objeto sin encerrarlo en una lista; es decir, la lista `supuestos_y_cuestiones` también admite directamente objetos de tipo string, `Supuesto` o `Cuestion`.

La combinación de distintas opciones crea el texto completo de una variante del problema.

Definición de la clase `ProblemaTipo`

```
class ProblemaTipo:
    def __init__(self, supuestos_y_cuestiones, setup=None):
        self.e = supuestos_y_cuestiones
        self.setup = setup

    <<Método __iter__ para la clase ProblemaTipo>>
    <<Método __next__ para la clase ProblemaTipo>>
```

El método `__iter__` inicializa el iterador configurando las estructuras internas, mientras que el método `__next__` obtiene la siguiente variante válida del problema, asegurándose de desechar en el proceso aquellas cuyas precondiciones no se satisfagan.

4.1.1. Método `__iter__(self)`

El atributo `self.l` es la lista `self.e` normalizada para que todos sus objetos sean listas; `self.long` es el número de componentes que constituyen el texto de cada variante. `self.i` es el iterador interno encargado de la combinatoria y `self.c` es un contador de intentos de generación.

Método `__iter__` para la clase `ProblemaTipo`

```
def __iter__(self):
    self.l = [x if isinstance(x, list) else [x] for x in self.e]
    self.long = len(self.l)
    self.i = iter(Marcador([len(x) for x in self.l]))
    self.c = 0
    return self
```

4.1.2. Método `__next__`

La generación de variantes de un `ProblemaTipo` se produce dentro de un bucle que solo se detiene cuando el intento de iterar el marcador interno, `next(self.i)`, lanza una excepción `StopIteration` por haberse agotado todas las combinaciones posibles.

En cada intento de generación:

1. El contador `self.c` se incrementa de forma consecutiva.
2. Las variables `enunciado`, `hipotesis` y `cuestiones` se inicializan vacías.
3. Se recorren uno a uno los componentes de la variante candidata.

Si todos los componentes superan con éxito sus respectivas validaciones, el método devuelve una tupla con el identificador del intento y las estructuras generadas: `(str(self.c), enunciado, cuestiones)`.

Método `__next__` para la clase `ProblemaTipo`

```
def __next__(self):
    self.c += 1
    while True:
        try:
            variante = next(self.i)
        except StopIteration:
            raise StopIteration

        ns = self.setup() if self.setup else {}
        enunciado = ""
        hipotesis = []
        cuestiones = []

        for n in range(self.long+1):
            if n == self.long:
                return (str(self.c), enunciado, cuestiones)

        componente = self.l[n][variante[n]]
        <<Tratamiento en función del tipo de componente>>
```

El fragmento de código `<<Tratamiento en función del tipo de componente>>` procesa el `componente` que corresponda en cada iteración del bucle `for` según su tipo:

- Cuando el `componente` es una cadena, se considera parte del enunciado del problema y se concatena al string `enunciado`.
- Cuando el `componente` es un objeto `Supuesto`, se testea su precondition. Si `componente.p` es `True` o no es refutable a partir de las `hipotesis` acumuladas, su texto `componente.e` se añade al enunciado y su significado lógico `componente.s` se incorpora a las `hipotesis`. En caso contrario, el análisis de la combinación actual se interrumpe de inmediato mediante un `break`, se notifica en pantalla el descarte y se pasa a evaluar el siguiente elemento del marcador.
- Cuando el `componente` es un objeto `Cuestion`, se testea su precondition. Si es `True` o no es refutable a partir de las `hipotesis` acumuladas, el par (`componente.e`, veracidad o falsedad de `componente.s`) se añade a la lista de `cuestiones`. En caso contrario, se imprime la notificación en pantalla y se interrumpe el procesamiento de la variante mediante un `break`; esto hace que la combinación entera quede descartada de inmediato, ignorando cualquier componente o lista de `cuestiones` posterior que restase por evaluar en esa iteración.

Tratamiento en función del tipo de componente

```
if isinstance(componente, str):
    enunciado = enunciado + _ns_interp(componente, ns)

elif isinstance(componente, Supuesto):
    precond = _ns_eval(componente.p, ns)
    if test(precond, hipotesis):
        enunciado = enunciado + _ns_interp(_ns_eval(componente.e, ns), ns)
        hipotesis = hipotesis + [_ns_eval(componente.s, ns)]
    else:
        print('\n Supuesto: ' + str(componente.e) \
```

```

        + ' rechazado por ' + _fuente_precond(componente) + '\n')
    break

elif isinstance(componente, Cuestion):
    precond = _ns_eval(componente.p, ns)
    semantica = _ns_eval(componente.s, ns)
    texto = _ns_interp(_ns_eval(componente.e, ns), ns)
    if test(precond, hipotesis):
        cuestiones = cuestiones + \
            [(texto, (True if test(semantica, hipotesis) else False), 1, componente.x)]
    else:
        cuestiones = cuestiones + \
            [(texto, 'rechazada por ' + _fuente_precond(componente), 0, componente.x)]
    print('\n Cuestion: ' + str(componente.e) \
        + ' rechazada por ' + _fuente_precond(componente) + '\n')
    break

```

4.2. Ejemplo de funcionamiento

Escribamos un hipotético ejercicio (es decir una lista que contiene cadenas de caracteres, supuestos, cuestiones, sublistas de supuestos o sublistas de cuestiones):

Ejemplo de programación de un ejercicio

```

ejercicio = [ "Considere ",
    [
        Supuesto("A ", v("A")),
        Supuesto("B ", v("B")),
    ],
    [
        Supuesto("y que A implica C. ", v("A") >> v("C")),
        Supuesto("y que B equivale a D. ", v("B") ** v("D")),
    ],
    "Conteste: ",
    [
        Cuestion("¿Se da C?", v("C"), exp="Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A"),
        Cuestion("¿Se da D?", v("D")),
        Cuestion("¿Se da E?", v("E"), v("D")),
    ],
    [
        Cuestion("¿Se dan A y C?", v("A") & v("C")),
        Cuestion("¿Se dan C y D?", v("C") & v("D")),
    ],
]

```

A continuación veamos lo que hace `ProblemaTipo` con la lista `ejercicio`. Observe que en cada iteración se toma un elemento de cada lista de supuestos y un elemento de cada lista de cuestiones para formar la combinación de un enunciado (compuesto por supuestos) y un subconjunto de preguntas (tantas como sublistas de cuestiones). Si alguna cuestión es descartada en una combinación, se salta a la siguiente combinación (indicando la cuestión descartada y el motivo). Así, `ProblemaTipo` muestra todas las combinaciones posibles de cuestiones que se pueden responder para cada combinación de supuestos. Fíjese que los elementos que no son listas (aquí las dos cadenas de caracteres) funcionan como sublistas con un único elemento (por eso en todas las iteraciones aparece tanto 'Considere' como 'Conteste:').

Ejemplo de ProblemaTipo

```

for i in ProblemaTipo( ejercicio ):
    print(i)

```

```

('1', 'Considere A y que A implica C. Conteste: ', [(('¿Se da C?', True, 1, 'Solo sabemos que C es cierto y si A implica C y
('2', 'Considere A y que A implica C. Conteste: ', [(('¿Se da C?', True, 1, 'Solo sabemos que C es cierto y si A implica C y
('3', 'Considere A y que A implica C. Conteste: ', [(('¿Se da D?', False, 1, ''), ('¿Se dan A y C?', True, 1, ''))])
('4', 'Considere A y que A implica C. Conteste: ', [(('¿Se da D?', False, 1, ''), ('¿Se dan C y D?', False, 1, ''))])

```

```

Cuestion: ¿Se da E? rechazada por v('D')

```

```

Cuestion: ¿Se da E? rechazada por v('D')

```

```

('5', 'Considere A y que B equivale a D. Conteste: ', [(('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica
('6', 'Considere A y que B equivale a D. Conteste: ', [(('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica
('7', 'Considere A y que B equivale a D. Conteste: ', [(('¿Se da D?', False, 1, ''), ('¿Se dan A y C?', False, 1, ''))])
('8', 'Considere A y que B equivale a D. Conteste: ', [(('¿Se da D?', False, 1, ''), ('¿Se dan C y D?', False, 1, ''))])

```

```

Cuestion: ¿Se da E? rechazada por v('D')

```

```

Cuestion: ¿Se da E? rechazada por v('D')

```

```
('9', 'Considere B y que A implica C. Conteste: ', [( '¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y '
('10', 'Considere B y que A implica C. Conteste: ', [( '¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y '
('11', 'Considere B y que A implica C. Conteste: ', [( '¿Se da D?', False, 1, ''), ( '¿Se dan A y C?', False, 1, '')])
('12', 'Considere B y que A implica C. Conteste: ', [( '¿Se da D?', False, 1, ''), ( '¿Se dan C y D?', False, 1, '')])
```

Question: ¿Se da E? rechazada por v('D')

Question: ¿Se da E? rechazada por v('D')

```
('13', 'Considere B y que B equivale a D. Conteste: ', [( '¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica '
('14', 'Considere B y que B equivale a D. Conteste: ', [( '¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica '
('15', 'Considere B y que B equivale a D. Conteste: ', [( '¿Se da D?', True, 1, ''), ( '¿Se dan A y C?', False, 1, '')])
('16', 'Considere B y que B equivale a D. Conteste: ', [( '¿Se da D?', True, 1, ''), ( '¿Se dan C y D?', False, 1, '')])
('17', 'Considere B y que B equivale a D. Conteste: ', [( '¿Se da E?', False, 1, ''), ( '¿Se dan A y C?', False, 1, '')])
('18', 'Considere B y que B equivale a D. Conteste: ', [( '¿Se da E?', False, 1, ''), ( '¿Se dan C y D?', False, 1, '')])
```

Cuando la lista de cuestiones es extensa (o incluye aclaraciones), la representación en una tupla puede extenderse más allá de los márgenes de este documento al utilizar `print`. Para mejorar la claridad y facilitar la comprensión del funcionamiento de `ProblemaTipo`, desempaquetaremos los componentes devueltos por el iterador y los presentaremos de forma estructurada. Esto permitirá visualizar mejor los resultados.

En cada iteración, presentaremos el enunciado completo correspondiente, junto con las cuestiones incluidas, cada una en una línea separada (indicando su veracidad y la explicación incluida en el atributo `self.x` si no la hemos dejado vacía). Si la variante fue abortada, se indicará cuál fue la cuestión desencadenante y el motivo de la interrupción.

```
for i in ProblemaTipo( ejercicio ):
    # Desempaquetamos la tupla en variables
    id_pregunta, EnunciadoCompleto, lista_Cuestiones = i

    # Imprimimos los primeros datos
    print(f"ID: {id_pregunta}")
    print(f"Enunciado completo: {EnunciadoCompleto}")

    # Imprimimos la lista de cuestiones, cada cuestión en una línea
    print(*lista_Cuestiones, sep='\n')
    print('\n')
```

ID: 1

Enunciado completo: Considere A y que A implica C. Conteste:

```
('¿Se da C?', True, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
('¿Se dan A y C?', True, 1, '')
```

ID: 2

Enunciado completo: Considere A y que A implica C. Conteste:

```
('¿Se da C?', True, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
('¿Se dan C y D?', False, 1, '')
```

ID: 3

Enunciado completo: Considere A y que A implica C. Conteste:

```
('¿Se da D?', False, 1, '')
('¿Se dan A y C?', True, 1, '')
```

ID: 4

Enunciado completo: Considere A y que A implica C. Conteste:

```
('¿Se da D?', False, 1, '')
('¿Se dan C y D?', False, 1, '')
```

Question: ¿Se da E? rechazada por v('D')

Question: ¿Se da E? rechazada por v('D')

ID: 5

Enunciado completo: Considere A y que B equivale a D. Conteste:

('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
('¿Se dan A y C?', False, 1, '')

ID: 6

Enunciado completo: Considere A y que B equivale a D. Conteste:

('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
('¿Se dan C y D?', False, 1, '')

ID: 7

Enunciado completo: Considere A y que B equivale a D. Conteste:

('¿Se da D?', False, 1, '')
('¿Se dan A y C?', False, 1, '')

ID: 8

Enunciado completo: Considere A y que B equivale a D. Conteste:

('¿Se da D?', False, 1, '')
('¿Se dan C y D?', False, 1, '')

Question: ¿Se da E? rechazada por v('D')

Question: ¿Se da E? rechazada por v('D')

ID: 9

Enunciado completo: Considere B y que A implica C. Conteste:

('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
('¿Se dan A y C?', False, 1, '')

ID: 10

Enunciado completo: Considere B y que A implica C. Conteste:

('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
('¿Se dan C y D?', False, 1, '')

ID: 11

Enunciado completo: Considere B y que A implica C. Conteste:

('¿Se da D?', False, 1, '')
('¿Se dan A y C?', False, 1, '')

ID: 12

Enunciado completo: Considere B y que A implica C. Conteste:

('¿Se da D?', False, 1, '')
('¿Se dan C y D?', False, 1, '')

Question: ¿Se da E? rechazada por v('D')

Question: ¿Se da E? rechazada por v('D')

ID: 13

Enunciado completo: Considere B y que B equivale a D. Conteste:

('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
('¿Se dan A y C?', False, 1, '')

ID: 14

Enunciado completo: Considere B y que B equivale a D. Conteste:

('¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
('¿Se dan C y D?', False, 1, '')

ID: 15

Enunciado completo: Considere B y que B equivale a D. Conteste:

('¿Se da D?', True, 1, '')

```
('¿Se dan A y C?', False, 1, '')
```

ID: 16

Enunciado completo: Considere B y que B equivale a D. Conteste:

```
('¿Se da D?', True, 1, '')
```

```
('¿Se dan C y D?', False, 1, '')
```

ID: 17

Enunciado completo: Considere B y que B equivale a D. Conteste:

```
('¿Se da E?', False, 1, '')
```

```
('¿Se dan A y C?', False, 1, '')
```

ID: 18

Enunciado completo: Considere B y que B equivale a D. Conteste:

```
('¿Se da E?', False, 1, '')
```

```
('¿Se dan C y D?', False, 1, '')
```

En el ejemplo anterior, se observa que, para cada combinación de supuestos, en cada iteración solo se presenta una cuestión de cada sublista de cuestiones. Dado que hay solo dos sublistas, esto resulta en la visualización de dos preguntas por iteración, esto dificulta la revisión y mantenimiento del banco de preguntas.

En el anterior ejemplo, y aunque existen únicamente cuatro enunciados posibles y cinco preguntas en total, la combinatoria ha generado 18 variantes no descartables, cada una con un subconjunto de solo dos preguntas. Esta complejidad complica la tarea de revisar y mantener un banco de ejercicios eficaz.

Capítulo 5

Clase ProblemaTipoProfe

Para facilitar la revisión y el mantenimiento de cada ejercicio, es preferible ver el conjunto completo de cuestiones para cada uno de los enunciados. La clase `ProblemaTipoProfe` lo hace, indicando en cada caso si son `True` o `False` (o si serán descartadas). Para ello utiliza el procedimiento auxiliar `CuestionesJuntas`, que crea una sublista con cada `Cuestion`. De este modo, como al iterar se toma un elemento de cada una de las sublistas, se mostrarán todas las cuestiones a la vez.

5.1. Implementación

Hay dos diferencias entre el código de la clase `ProblemaTipoProfe` y el la clase `ProblemaTipo`.

- La primera es que en `self.e` cada `Cuestion` aparece como único elemento de una sublista, para así lograr que se ven todas las cuestiones en cada iteración.
- La segunda diferencia se produce en la definición del método `__next__`, que aquí no oculta las cuestiones que sí son rechazadas cuando usamos `ProblemaTipo`.

Definición de la clase `ProblemaTipoProfe`

```
class ProblemaTipoProfe:
    def __init__(self, supuestos_y_cuestiones, setup=None):
        self.e = CuestionesJuntas(supuestos_y_cuestiones)
        self.setup = setup

    <<Método __iter__ para la clase ProblemaTipo>>
    <<Método __next__ para la clase ProblemaTipoProfe>>
```

5.1.1. Método `__next__` para la clase `ProblemaTipoProfe`

La diferencia con respecto al <<Método `__next__` para la clase `ProblemaTipo`>> es el tratamiento de las componentes; no se excluyen las preguntas que serán rechazadas en la versión para el estudiante, se añade la indicación 'rechazada por' y la precondition que ocasionará el rechazo con la clase `ProblemaTipo`.

Método `__next__` para la clase `ProblemaTipoProfe`

```
def __next__(self):
    self.c += 1
    while True:
        try:
            variante = next(self.i)
        except StopIteration:
            raise StopIteration

    ns = self.setup() if self.setup else {}
    enunciado = ""
    hipotesis = []
    cuestiones = []

    for n in range(self.long+1):
        if n == self.long:
            return (str(self.c), enunciado, cuestiones)

    componente = self.l[n][variante[n]]
    <<Tratamiento en función del tipo de componente para la versión del Profe>>
```

Tratamiento en función del tipo de componente en la versión del Profe

Tratamiento en función del tipo de componente para la versión del Profe

```
if isinstance(componente, str):
    enunciado = enunciado + _ns_interp(componente, ns)

elif isinstance(componente, Supuesto):
    precond = _ns_eval(componente.p, ns)
    if test(precond, hipotesis):
        enunciado = enunciado + _ns_interp(_ns_eval(componente.e, ns), ns)
        hipotesis = hipotesis + [_ns_eval(componente.s, ns)]
    else:
        print('\n Supuesto: ' + str(componente.e) \
              + ' rechazado por ' + _fuente_precond(componente) + '\n')
        break

elif isinstance(componente, Cuestion):
    precond = _ns_eval(componente.p, ns)
    semantica = _ns_eval(componente.s, ns)
    texto = _ns_interp(_ns_eval(componente.e, ns), ns)
    if test(precond, hipotesis):
        cuestiones = cuestiones + \
            [(texto, (True if test(semantica, hipotesis) else False), 1, componente.x)]
    else:
        cuestiones = cuestiones + \
            [(texto, 'rechazada por ' + _fuente_precond(componente), 0)]
```

5.1.2. Función auxiliar CuestionesJuntas

Este bloque de código define la función `CuestionesJuntas(lista)`, cuyo objetivo principal es **aislar cada objeto de la clase `Cuestion` dentro de su propia sublista individual**, manteniendo otros tipos de datos (como cadenas o listas de supuestos) intactos.

En detalle:

1. **CreaLista(t)**: Una función auxiliar que asegura que cualquier elemento sea tratado como una lista (si no lo es, lo envuelve en una para poder indexarlo).
2. **Iteración**: Recorre cada elemento `e` de la lista de entrada `lista`.
3. **Strings y No-Cuestiones**: Si el elemento es una cadena o si el primer elemento de su contenido (tras pasarlo por `CreaLista`) no es de tipo `Cuestion` (por ejemplo, un objeto de la clase `Supuesto`), se añade directamente a la lista resultante `p` mediante `append`.
4. **Aislamiento de Cuestion**: Si el elemento es una instancia de `Cuestion` o pertenece a una lista de cuestiones, se utiliza `extend(CreaLista(e))`, lo que descompone cualquier agrupación previa de `cuestiones`, añadiendo cada `Cuestion` como una sublista independiente (y con un único elemento) dentro de `p`.

En resumen: Transforma una lista mixta de modo que cada objeto `Cuestion` acabe confinado en una sublista propia. De este modo, el generador multidimensional las procesa de forma independiente y simultánea.

Objetivo: Esto permite que la clase `ProblemaTipoProfe` muestre todas las cuestiones posibles para cada enunciado en lugar de una sola.

Metodo auxiliar para juntar cuestiones en formato para profe

```
def CuestionesJuntas(lista):
    def CreaLista(t):
        return t if isinstance(t, list) else [t]
    p = []
    for e in lista:
        if isinstance(e, str):
            p.append(e)
        elif not isinstance(CreaLista(e)[0], Cuestion):
            p.append(e)
        else:
            p.extend(CreaLista(e))
    return p
```

5.2. Ejemplo de funcionamiento

Usaremos la misma lista `ejercicio` que usamos en el capítulo anterior

```
for i in ProblemaTipoProfe( ejercicio ):
    print(i)
```

```
('1', 'Considere A y que A implica C. Conteste: ', [( '¿Se da C?', True, 1, 'Solo sabemos que C es cierto y si A implica C y
( '2', 'Considere A y que B equivale a D. Conteste: ', [( '¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica
( '3', 'Considere B y que A implica C. Conteste: ', [( '¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y
( '4', 'Considere B y que B equivale a D. Conteste: ', [( '¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica
```

Como la vez anterior la representación en una tupla usando `print` se ha extendido más allá de los márgenes de este documento. Para evitar este problema y que el lector vea mejor el funcionamiento de `ProblemaTipoProfe`, vamos usar el mismo truco que en el capítulo anterior: Debajo de cada variante del enunciado completo, mostraremos todas las cuestiones de manera individual, cada una en una línea separada, junto con el cálculo de su veracidad o la indicación de si serán descartadas al emplear la clase `ProblemaTipo`. Si la cuestión incluye el atributo `self.x` con una explicación, también se muestra la explicación (si no, aparece una cadena vacía `' '`).

```
for i in ProblemaTipoProfe( ejercicio ):
    # Desempaquetamos la tupla en variables
    id_pregunta, EnunciadoCompleto, lista_Cuestiones = i

    # Imprimimos los primeros datos
    print(f"ID: {id_pregunta}")
    print(f"Enunciado completo: {EnunciadoCompleto}")

    # Imprimimos la lista de cuestiones, cada cuestión en una línea
    print(*lista_Cuestiones, sep='\n')
    print('\n')
```

ID: 1

Enunciado completo: Considere A y que A implica C. Conteste:

```
( '¿Se da C?', True, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
( '¿Se da D?', False, 1, '' )
( '¿Se da E?', "rechazada por v('D')", 0)
( '¿Se dan A y C?', True, 1, '' )
( '¿Se dan C y D?', False, 1, '' )
```

ID: 2

Enunciado completo: Considere A y que B equivale a D. Conteste:

```
( '¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
( '¿Se da D?', False, 1, '' )
( '¿Se da E?', "rechazada por v('D')", 0)
( '¿Se dan A y C?', False, 1, '' )
( '¿Se dan C y D?', False, 1, '' )
```

ID: 3

Enunciado completo: Considere B y que A implica C. Conteste:

```
( '¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
( '¿Se da D?', False, 1, '' )
( '¿Se da E?', "rechazada por v('D')", 0)
( '¿Se dan A y C?', False, 1, '' )
( '¿Se dan C y D?', False, 1, '' )
```

ID: 4

Enunciado completo: Considere B y que B equivale a D. Conteste:

```
( '¿Se da C?', False, 1, 'Solo sabemos que C es cierto y si A implica C y además sabemos que ocurre A')
( '¿Se da D?', True, 1, '' )
( '¿Se da E?', False, 1, '' )
( '¿Se dan A y C?', False, 1, '' )
( '¿Se dan C y D?', False, 1, '' )
```

Vemos que son cuatro enunciados en total, y para cada uno de ellos vemos lo qué pasa con cada una de las cinco cuestiones posibles: si son verdaderas, si son falsas o si serán descartadas (y el motivo). Ahora es mucho más fácil mejorar y depurar el `ejercicio` que estamos diseñando para nuestros exámenes de opción múltiple.

Capítulo 6

Clase ProblemaVF

A diferencia de las clases anteriores, que exploran de forma sistemática y secuencial todas las combinaciones posibles de un espacio combinatorio, la clase `ProblemaVF` está diseñada para generar ejercicios basados en un **banco de cuestiones aleatorizado**.

Su objetivo primordial es construir variantes de exámenes del tipo **Verdadero/Falso**, donde el enunciado permanece fijo y cada variante ofrece un subconjunto único y desordenado de preguntas seleccionadas al azar a partir de un repertorio común.

6.1. Implementación

El núcleo de esta clase radica en el uso de la función `sample` del módulo `random` de Python, lo que permite extraer muestras sin repetición del banco de preguntas en cada iteración.

Una particularidad fundamental de este diseño es que el método `__next__` es que no sigue un barrido secuencial de todas las combinaciones posibles. Aquí el iterador generará una variante al azar cada vez que se invoque el método `next()` (obviamente, dado que el banco de cuestiones es finito, aparecerán variantes repetidas de cuando en cuando).

Definición de la clase `ProblemaVF`

```
from random import sample
class ProblemaVF():
    def __init__(self, enunciado, cuestiones, NumPreguntas):
        self.e = enunciado
        self.c = cuestiones
        self.NumPreguntas = NumPreguntas

    def __iter__(self):
        self.contador = 0
        return self

    def __next__(self):
        cuestiones = sample(self.c, self.NumPreguntas)
        self.contador += 1
        return (str(self.contador), self.e, cuestiones)
```

6.1.1. Métodos de la clase `ProblemaVF`

- **`__init__(self, enunciado, cuestiones, NumPreguntas)`**: Inicializa el generador almacenando el texto base del problema (`enunciado`), la lista completa de tuplas con las preguntas y sus valores de verdad (`cuestiones`), y la cantidad exacta de ítems que se deben extraer para cada ejercicio (`NumPreguntas`).
- **`__iter__(self)`**: Prepara el objeto para ser iterado, inicializando un `self.contador` interno a cero que servirá para identificar unívocamente el ID de cada variante generada.
- **`__next__(self)`**: Constituye el motor de la clase. En cada invocación ejecuta tres pasos:
 1. Utiliza `sample(self.c, self.NumPreguntas)` para obtener una lista con el número de preguntas solicitado, garantizando que en una misma variante no se repita ninguna cuestión.

2. Incrementa el contador de variantes.
3. Devuelve una tupla con el identificador en formato de cadena, el enunciado estático y la lista aleatoria de preguntas seleccionadas con sus respectivos estados lógicos (**True/False**).

Nota de diseño: Dado que no existe una condición de parada intrínseca (como un extremo de lista), el bucle que consuma este iterador debe limitarse explícitamente desde el exterior (por ejemplo, mediante un bucle **for** con un rango acotado o controlando el número de peticiones).

6.2. Ejemplo de funcionamiento

A continuación se ilustra el comportamiento de la clase utilizando un banco de diez enunciados cotidianos relacionados con la vida académica.

```
banco = [
    ("Todo alumno que va a clase aprueba", False),
    ("Todo alumno que suspende va a clase", False),
    ("Todo alumno que sabe aprueba", True),
    ("Todo alumno que NO sabe suspende", False),
    ("Algunos alumnos aprueban copiando", True),
    ("Copiar es intolerable", True),
    ("Si aprueban todos es estupendo", True),
    ("Si aprueban todos eres buen profesor", False),
    ("Si suspenden todos eres mal profesor", False),
    ("Si suspenden todos es frustrante", True),]
```

6.2.1. Generación de cuatro variantes cortas

En este primer ensayo se configuran cuatro ejercicios, solicitando únicamente dos preguntas tipo verdadero/falso para cada uno.

```
enunciado = "Marque las verdaderas:"
```

Ejemplo de ProblemaVF

```
p = ProblemaVF (enunciado, banco, 2)      # Dos preguntas del banco por variante
p0 = iter(p)

for i in range(4):                        # Generamos cuatro variantes y paramos
    print(next(p0))
```

```
('1', 'Marque las verdaderas:', [('Si suspenden todos es frustrante', True), ('Si aprueban todos es estupendo', True)])
('2', 'Marque las verdaderas:', [('Copiar es intolerable', True), ('Todo alumno que suspende va a clase', False)])
('3', 'Marque las verdaderas:', [('Todo alumno que suspende va a clase', False), ('Si suspenden todos eres mal profesor', False)])
('4', 'Marque las verdaderas:', [('Todo alumno que suspende va a clase', False), ('Todo alumno que sabe aprueba', True)])
```

6.2.2. Generación de cinco variantes con presentación formateada

Como la representación en una sola tupla puede dificultar la lectura a medida que el número de ítems crece, vamos a desempaquetar los componentes devueltos por el iterador para volcar los datos de manera estructurada para que se vea mejor el resultado, pues vamos a mostrar cada cuestión en una línea independiente.

Dado que ahora lo vamos a ver mejor, aumentamos la complejidad solicitando tres preguntas por variante para un total de cinco ejercicios:

```
p = ProblemaVF (enunciado, banco, 3)      # Tres preguntas por variante
p0 = iter(p)

for i in range(5):
    # Desempaquetamos la tupla en variables
    id_pregunta, EnunciadoCompleto, lista_Cuestiones = next(p0)

    # Imprimimos los primeros datos
    f"ID: {id_pregunta}"
    print(f"Enunciado completo: {EnunciadoCompleto}")

    # Imprimimos la lista de cuestiones, cada cuestión en una línea
    print(*lista_Cuestiones, sep='\n')
    print('\n')
```

```
Enunciado completo: Marque las verdaderas:
('Copiar es intolerable', True)
```

```
('Si suspenden todos eres mal profesor', False)
('Si suspenden todos es frustrante', True)
```

```
Enunciado completo: Marque las verdaderas:
('Todo alumno que suspende va a clase', False)
('Si aprueban todos es estupendo', True)
('Si aprueban todos eres buen profesor', False)
```

```
Enunciado completo: Marque las verdaderas:
('Si suspenden todos eres mal profesor', False)
('Copiar es intolerable', True)
('Si aprueban todos es estupendo', True)
```

```
Enunciado completo: Marque las verdaderas:
('Si suspenden todos es frustrante', True)
('Todo alumno que suspende va a clase', False)
('Todo alumno que va a clase aprueba', False)
```

```
Enunciado completo: Marque las verdaderas:
('Todo alumno que NO sabe suspende', False)
('Todo alumno que suspende va a clase', False)
('Todo alumno que sabe aprueba', True)
```

Como se puede apreciar en el resultado impreso, cada uno de los cinco bloques representa un ejercicio completamente autónomo con un identificador secuencial, conservando el enunciado general e integrando un muestreo dinámico y heterogéneo extraído directamente de la lista de afirmaciones almacenada con el nombre de **banco**.

Capítulo 7

Clase SubPregunta y ProblemaMultiParte

Los formatos AMC y Moodle permiten agrupar varias sub-preguntas de opción múltiple bajo un mismo enunciado general. Cada sub-pregunta tiene su propio pequeño intro y su propio bloque de opciones (correctas e incorrectas). Este tipo de pregunta se llama *multi-parte* (AMC) o *cloze* (Moodle).

SubPregunta es un contenedor simple: un texto introductorio y una lista de **Cuestion** que se evaluarán todas (no se filtra por alternativa como en **ProblemaTipo**, sino que se muestran todas con su marca correcto/incorrecto).

ProblemaMultiParte sigue la misma lógica de iteración que **ProblemaTipo**:

- **componentes** es la lista con strings y **Supuesto** (o listas de ellos) que forma el enunciado común y determina las hipótesis activas.
- **subpreguntas** es la lista de **SubPregunta**, cada una con un intro y sus cuestiones.
- Por cada combinación de supuestos se produce una variante: el iterador devuelve (**etiqueta**, **enunciado_comun**, [(**intro1**, **cuestiones1**), (**intro2**, **cuestiones2**), ...]).

La diferencia clave respecto a **ProblemaTipo** es que el iterador usa `for...else` para separar limpiamente el bloque de construcción del enunciado (que puede abortar si un **Supuesto** es rechazado) del bloque de evaluación de las sub-preguntas (que solo se alcanza si el enunciado es válido).

Definición de SubPregunta y ProblemaMultiParte

```
class SubPregunta:
    def __init__(self, intro, cuestiones):
        self.intro = intro
        self.cuestiones = cuestiones if isinstance(cuestiones, list) else [cuestiones]

class ProblemaMultiParte:
    def __init__(self, componentes, subpreguntas, setup=None):
        """
        componentes: list de str / Supuesto / list de Supuesto (enunciado común)
        subpreguntas: list de SubPregunta
        setup: callable opcional
        """
        self.componentes = componentes
        self.subpreguntas = subpreguntas
        self.setup = setup

    def __iter__(self):
        self.l = [x if isinstance(x, list) else [x] for x in self.componentes]
        self.long = len(self.l)
        self.i = iter(Marcador([len(x) for x in self.l]))
        self.c = 0
        return self

    def __next__(self):
        self.c += 1
        while True:
            try:
                variante = next(self.i)
            except StopIteration:
                raise StopIteration
```

```

ns = self.setup() if self.setup else {}
enunciado = ""
hipotesis = []

for n in range(self.long):
    componente = self.l[n][variante[n]]
    if isinstance(componente, str):
        enunciado += _ns_interp(componente, ns)
    elif isinstance(componente, Supuesto):
        precond = _ns_eval(componente.p, ns)
        if test(precond, hipotesis):
            enunciado += _ns_interp(_ns_eval(componente.e, ns), ns)
            hipotesis = hipotesis + [_ns_eval(componente.s, ns)]
        else:
            print('\n Supuesto: ' + str(componente.e)
                  + ' rechazado por ' + _fuente_precond(componente) + '\n')
            break
    else:
        subpregs = []
        for sp in self.subpreguntas:
            intro_text = _ns_interp(sp.intro, ns)
            cuestiones = []
            for c in sp.cuestiones:
                precond = _ns_eval(c.p, ns)
                semantica = _ns_eval(c.s, ns)
                texto = _ns_interp(_ns_eval(c.e, ns), ns)
                if test(precond, hipotesis):
                    correcto = True if test(semantica, hipotesis) else False
                    cuestiones.append((texto, correcto, 1, c.x))
            subpregs.append((intro_text, cuestiones))
        return (str(self.c), enunciado, subpregs)

```

7.1. Ejemplo

```

from qbank import *

p = ProblemaMultiParte(
    componentes=[
        'Una desviación típica es un indicador ',
        [Supuesto('de dispersión. ', v('Disp')),
         Supuesto('de tendencia central. ', -v('Disp'))],
    ],
    subpreguntas=[
        SubPregunta('(en cuanto a su objetivo)',
            [Cuestion('de tendencia central', -v('Disp')),
             Cuestion('de dispersión', v('Disp'))]),
        SubPregunta('(en cuanto a su sensibilidad)',
            [Cuestion('sensible a valores extremos', v('Disp')),
             Cuestion('no muy sensible a extremos', -v('Disp'))]),
    ]
)

for etiqueta, enunciado, subpreguntas in p:
    print(f'=== Variante {etiqueta} ===')
    print(f'Enunciado: {enunciado}')
    for intro, cuestiones in subpreguntas:
        print(f' {intro}')
        for texto, correcto, _, _ in cuestiones:
            print(f' {" " if correcto else " "} {texto}')

```

```

=== Variante 1 ===
Enunciado: Una desviación típica es un indicador de dispersión.
(en cuanto a su objetivo)
de tendencia central
de dispersión
(en cuanto a su sensibilidad)
sensible a valores extremos
no muy sensible a extremos

=== Variante 2 ===
Enunciado: Una desviación típica es un indicador de tendencia central.
(en cuanto a su objetivo)
de tendencia central
de dispersión
(en cuanto a su sensibilidad)
sensible a valores extremos
no muy sensible a extremos

```

Parte II

Serialización y persistencia en formato JSON

El módulo `qbank._json` permite guardar y cargar objetos `ProblemaTipo` y `ProblemaVF` en ficheros JSON. Esto facilita compartir bancos de preguntas, o modificarlos en distintas mediante un editor visual sin tener que reescribir el código Python en cada ocasión.

Estructura del módulo qbank._json

El módulo completo está compuesto por varios fragmentos de código que se describen en las subsecciones siguientes.

```
import json as _json
import calcprop as _calcprop_mod
from qbank._quiz import (Supuesto, Cuestion, ProblemaTipo, ProblemaVF,
                          SubPregunta, ProblemaMultiParte)

__all__ = [
    'problema_from_dict',
    'problema_to_dict',
    'load_problema',
    'save_problema',
    'load_banco',
    'save_banco',
    'problema_to_python',
    'save_problema_py',
]

<<Espacio de nombres calcprop para JSON>>
<<Evaluación de expresiones JSON>>
<<Función setup desde string>>
<<Componente desde dict JSON>>
<<Slot desde JSON>>
<<Slot a JSON>>
<<Problema desde dict JSON>>
<<Problema a dict JSON>>
<<Cargar y guardar problema JSON>>
<<Cargar y guardar banco JSON>>
<<Exportación a código Python>>
```

Capítulo 8

Formato JSON

8.1. Para un ProblemaTipo

Un ProblemaTipo se representa con la siguiente estructura:

```
{
  "version": "1",
  "tipo": "ProblemaTipo",
  "nombre": "identificador_opcional",
  "setup": null,
  "componentes": [
    "texto fijo del enunciado",
    {"tipo": "Supuesto", "enunciado": "A ", "semantica": "v('A')", "precond": "True"},
    [
      {"tipo": "Supuesto", "enunciado": "alt1 ", "semantica": "v('A')"},
      {"tipo": "Supuesto", "enunciado": "alt2 ", "semantica": "v('B')"}
    ],
    [
      {"tipo": "Cuestion", "enunciado": "¿A?", "semantica": "v('A')",
       "precond": "True", "exp": ""}
    ]
  ]
}
```

El campo `componentes` es una lista cuyos elementos pueden ser:

- **Cadena de texto:** texto fijo que aparece en todas las variantes.
- **Dict:** un único Supuesto o Cuestion (equivale a una lista de un elemento).
- **Lista de dicts:** varias alternativas de las que ProblemaTipo elige una por variante.

Los campos `semantica` y `precond` se almacenan como **cadenas de texto**. Al deserializar¹ se evalúan con `eval()` en el espacio de nombres (*namespace*)² de `calccprop`, lo que permite expresiones como `v('A') & v('B')`, los booleanos literales `True` y `False`, o lambdas para semánticas paramétricas (`"lambda ns: ns['x'] > 0"`).

¹«Serializar» significa convertir un objeto Python en memoria (un ProblemaTipo, por ejemplo) a un formato que se puede guardar o transmitir —en este caso, JSON. «Deserializar» es la operación inversa: leer ese JSON y reconstruir el objeto Python.

²«Espacio de nombres» es la traducción literal de *namespace*, que en Python significa simplemente el conjunto de variables (nombres y valores) disponibles en un momento dado. Aquí los usaremos en dos contextos distintos: Por una parte en `_EVAL_NS`; es decir, el diccionario que contiene todas las funciones y variables del módulo `calccprop` (`v`, `test`, etc.). Cuando se evalúa la cadena `v('A') & v('B')` con `eval()`, se le pasa ese diccionario para que `v` esté disponible. Aquí «espacio de nombres» podría sustituirse por «entorno de evaluación» o simplemente «las funciones de `calccprop` disponibles para `eval()`». Y por otra parte con el dict que devuelve `setup()` — por ejemplo `{'a': 3, 'b': 7, 'suma': 10}`. Se llamará «espacio de nombres de la variante» porque es el conjunto de variables que ese problema concreto pone a disposición de los textos (`@a`, `@b`) y las semánticas (`lambda ns: ns['a'] + ns['b'] > 10`). Aquí el término más claro sería simplemente «diccionario de variables» o «variables de la variante».

8.2. Para un ProblemaVF

Un ProblemaVF tiene un formato más simple:

```
{
  "version": "1",
  "tipo": "ProblemaVF",
  "nombre": "identificador_opcional",
  "enunciado": "Marque las verdaderas:",
  "NumPreguntas": 3,
  "cuestiones": [
    {"texto": "Afirmación 1", "respuesta": true},
    {"texto": "Afirmación 2", "respuesta": false}
  ]
}
```

Un banco con múltiples problemas agrupa varios dicts bajo la clave "**banco**":

```
{
  "version": "1",
  "banco": [ {problema1}, {problema2}, ... ]
}
```

Capítulo 9

Evaluación de expresiones en el espacio de nombres de `calcprop`

Las semánticas y precondiciones se guardan en JSON como cadenas de texto y se recuperan evaluándolas con `eval()` en el espacio de nombres del módulo `calcprop`. Así, expresiones como `v('A') & v('B')` se resuelven correctamente al deserializar.

El espacio de nombres se construye una sola vez al importar el módulo:

```
_EVAL_NS = vars(_calcprop_mod).copy()
```

La función `_eval_expr` gestiona los casos especiales (booleanos literales) antes de delegar en `eval`, evitando así que `"True"` se evalúe como el nombre Python `True` solo si el usuario ha escrito exactamente esa cadena:

```
def _eval_expr(expr):
    """Evalúa una expresión (string o bool) en el namespace de calcprop."""
    if isinstance(expr, bool):
        return expr
    if expr == "True":
        return True
    if expr == "False":
        return False
    return eval(expr, _EVAL_NS)
```

Capítulo 10

Función de setup desde código Python

El campo `setup` permitirá realizar cálculos con python para dar valores a preguntas con parámetros.

Cuando el campo `setup` no es `null`, contiene el código Python (como cadena) que se ejecuta al comienzo de cada iteración de `ProblemaTipo`. La función `_make_setup_fn` lo convierte en un callable que devuelve el diccionario de variables definidas, excluyendo las que empiezan por guión bajo (convención para internas de Python):

Función setup desde string

```
def _make_setup_fn(code_str):  
    """Crea un callable setup() a partir de código Python en string."""  
    def setup():  
        ns = {}  
        exec(code_str, ns)  
        return {k: v for k, v in ns.items() if not k.startswith('_')}  
    return setup
```

Por ejemplo, si el JSON contiene:

```
"setup": "import random\nnx = random.randint(1, 9)\ny = random.randint(1, 9)"
```

en cada iteración se ejecuta ese código y las variables `x` e `y` quedan disponibles para interpolar en los textos del problema mediante `@x` e `@y`.

En el ejemplo, `\n` indica un salto de línea en el script de python; es decir. el `setup` anterior se despliega en el script:

```
import random  
x = random.randint(1, 9)  
y = random.randint(1, 9)
```

Capítulo 11

Deserialización de componentes

La función `_componente_from_dict` crea un objeto `Supuesto` o `Cuestion` a partir de un dict JSON. Además de evaluar la semántica y la precondition, adjunta al objeto un atributo `_json` con los datos originales en formato de cadena. Ese atributo permite la serialización inversa (`problema_to_dict`) sin pérdida de información: el valor de cadena de la expresión se conserva tal como fue escrito por el usuario, sin que la evaluación lo destruya.

Componente desde dict JSON

```
def _componente_from_dict(d):
    tipo = d.get("tipo")
    enunciado = d["enunciado"]
    semantica_str = d["semantica"]
    precondition = d.get("precond", "True")

    semantica = _eval_expr(semantica_str)
    precondition = _eval_expr(precondition_str)

    if tipo == "Supuesto":
        obj = Supuesto(enunciado, semantica, precondition)
        obj._json = {"tipo": "Supuesto", "enunciado": enunciado,
                    "semantica": semantica_str, "precond": precondition_str}
    elif tipo == "Cuestion":
        exp = d.get("exp", "")
        obj = Cuestion(enunciado, semantica, precondition, exp)
        obj._json = {"tipo": "Cuestion", "enunciado": enunciado,
                    "semantica": semantica_str, "precond": precondition_str, "exp": exp}
    else:
        raise ValueError(f"Tipo de componente desconocido: {tipo!r}")
    return obj
```

La función `_slot_from_json` despacha en función del tipo de cada elemento del slot (cadena, dict único o lista de dicts):

Slot desde JSON

```
def _slot_from_json(slot):
    if isinstance(slot, str):
        return slot
    elif isinstance(slot, dict):
        return _componente_from_dict(slot)
    elif isinstance(slot, list):
        return [_slot_from_json(item) for item in slot]
    else:
        raise ValueError(f"Componente JSON inválido: {slot!r}")
```

Capítulo 12

Serialización de componentes

La función `_slot_to_json` recorre el problema en sentido inverso. Cuando el objeto tiene el atributo `_json` (cargado desde JSON), lo devuelve directamente. Cuando el objeto fue creado a mano en Python, usa `repr()` sobre la semántica y la precondition: `repr()` de cualquier fórmula `calcprop` produce exactamente la cadena que `_eval_expr` sabe leer (p.ej. `v('A') >> v('C')`), por lo que el round-trip es transparente. Esta propiedad permite serializar problemas escritos directamente en Python sin modificarlos.

La función auxiliar `_expr_to_str` gestiona el caso especial de los callables (lambdas de `setup` paramétrico), cuyo código fuente no es recuperable desde el objeto función:

Slot a JSON

```
def _expr_to_str(val, campo):
    """Convierte una semántica o precondition al string JSON equivalente.

    Usa repr() sobre objetos calcprop (cuyo repr es evaluable con _eval_expr).
    Falla con un mensaje claro si val es un callable (lambda de setup paramétrico).
    """
    if callable(val):
        raise ValueError(
            f"El campo '{campo}' es un callable (lambda). "
            "Los componentes con semánticas o precondiciones lambda solo pueden "
            "serializarse si fueron cargados con load_problema() o problema_from_dict().")
    return repr(val)

def _slot_to_json(slot):
    if isinstance(slot, str):
        return slot
    elif isinstance(slot, (Supuesto, Cuestion)):
        if hasattr(slot, '_json'):
            return slot._json
        # Fallback para objetos creados directamente en Python:
        # repr() de cualquier fórmula calcprop es evaluable por _eval_expr.
        d = {
            'tipo': 'Supuesto' if isinstance(slot, Supuesto) else 'Cuestion',
            'enunciado': slot.e,
            'semantica': _expr_to_str(slot.s, 'semantica'),
            'precond': _expr_to_str(slot.p, 'precond'),
        }
        if isinstance(slot, Cuestion):
            d['exp'] = slot.x
        return d
    elif isinstance(slot, list):
        return [_slot_to_json(item) for item in slot]
    else:
        raise ValueError(f"Tipo no serializable: {type(slot)}")
```

Capítulo 13

Serialización de problemas completos

Las funciones públicas `problema_from_dict` y `problema_to_dict` realizan la conversión bidireccional entre dicts Python y objetos `ProblemaTipo` / `ProblemaVF`.

`problema_from_dict` distingue el tipo de problema por el campo "tipo" del dict, reconstruye los componentes llamando a `_slot_from_json` para cada uno, y convierte el campo "setup" (si no es null) en un callable mediante `_make_setup_fn`. Los metadatos `_nombre` y `_setup_str` se guardan como atributos del objeto para que `problema_to_dict` pueda recuperarlos sin necesidad de solicitarlos de nuevo al usuario.

```
def problema_from_dict(d):
    """Crea un ProblemaTipo o ProblemaVF a partir de un dict JSON."""
    tipo = d.get("tipo")

    if tipo == "ProblemaTipo":
        componentes = [_slot_from_json(s) for s in d["componentes"]]
        setup_str = d.get("setup")
        setup = _make_setup_fn(setup_str) if setup_str else None
        p = ProblemaTipo(componentes, setup=setup)
        p._nombre = d.get("nombre", "")
        p._setup_str = setup_str
        return p

    elif tipo == "ProblemaMultiParte":
        componentes = [_slot_from_json(s) for s in d["componentes"]]
        subpreguntas = [
            SubPregunta(
                sp["intro"],
                [_componente_from_dict(c) for c in sp["cuestiones"]]
            )
            for sp in d["subpreguntas"]
        ]
        setup_str = d.get("setup")
        setup = _make_setup_fn(setup_str) if setup_str else None
        p = ProblemaMultiParte(componentes, subpreguntas, setup=setup)
        p._nombre = d.get("nombre", "")
        p._setup_str = setup_str
        return p

    elif tipo == "ProblemaVF":
        enunciado = d["enunciado"]
        cuestiones = [(c["texto"], c["respuesta"]) for c in d["cuestiones"]]
        num = d["NumPreguntas"]
        p = ProblemaVF(enunciado, cuestiones, num)
        p._nombre = d.get("nombre", "")
        return p

    else:
        raise ValueError(f"Tipo de problema desconocido: {tipo!r}")
```

`problema_from_dict` reconstruye los tres tipos de problema según la clave `tipo` del dict: `ProblemaTipo`, `ProblemaMultiParte` (con sus `subpreguntas`, cada una una lista de `Cuestion`) y `ProblemaVF`.

La función inversa, `problema_to_dict`, admite siempre objetos `ProblemaVF` (sus datos son directamente serializables). Para `ProblemaTipo` y `ProblemaMultiParte` delega en `_slot_to_json`: los componentes pueden haber sido cargados desde JSON (con atributo `_json`) o creados directamente en Python (en cuyo caso `_slot_to_json` usa `repr()`); en el caso multi-parte serializa además cada `SubPregunta` con

su intro y sus cuestiones. La única restricción es que los `setup` definidos como funciones Python no se pueden serializar (su código fuente no es introspectable); en ese caso se lanza un error claro.

```

def problema_to_dict(problema):
    """Serializa un ProblemaTipo o ProblemaVF a dict.

    ProblemaTipo con componentes creados directamente en Python se serializa
    usando repr() sobre las fórmulas calcprop. Si alguna semántica o precondition
    es un callable (lambda de setup paramétrico), la serialización falla.
    Los setup definidos como funciones Python (no cargados desde JSON) no pueden
    serializarse; en ese caso se lanza un error. ProblemaVF siempre puede serializarse.
    """
    if isinstance(problema, ProblemaTipo):
        componentes = [_slot_to_json(s) for s in problema.e]
        setup_str = getattr(problema, '_setup_str', None)
        if setup_str is None and problema.setup is not None:
            raise ValueError(
                "ProblemaTipo tiene un setup callable pero no fue creado desde JSON. "
                "No es posible serializar un setup definido directamente en Python. "
                "Escribe el código del setup como cadena y carga el problema con "
                "problema_from_dict().")
        return {
            "version": "1",
            "tipo": "ProblemaTipo",
            "nombre": getattr(problema, '_nombre', ''),
            "setup": setup_str,
            "componentes": componentes,
        }
    elif isinstance(problema, ProblemaMultiParte):
        componentes = [_slot_to_json(s) for s in problema.componentes]
        setup_str = getattr(problema, '_setup_str', None)
        if setup_str is None and problema.setup is not None:
            raise ValueError(
                "ProblemaMultiParte tiene un setup callable pero no fue creado desde JSON. "
                "Escribe el código del setup como cadena y carga el problema con "
                "problema_from_dict().")
        subpreguntas = []
        for sp in problema.subpreguntas:
            cuestiones_json = []
            for c in sp.cuestiones:
                if hasattr(c, '_json'):
                    cuestiones_json.append(c._json)
                else:
                    cuestiones_json.append({
                        'tipo': 'Cuestion',
                        'enunciado': c.e,
                        'semantica': _expr_to_str(c.s, 'semantica'),
                        'precond': _expr_to_str(c.p, 'precond'),
                        'exp': c.x,
                    })
            subpreguntas.append({"intro": sp.intro, "cuestiones": cuestiones_json})
        return {
            "version": "1",
            "tipo": "ProblemaMultiParte",
            "nombre": getattr(problema, '_nombre', ''),
            "setup": setup_str,
            "componentes": componentes,
            "subpreguntas": subpreguntas,
        }
    elif isinstance(problema, ProblemaVF):
        return {
            "version": "1",
            "tipo": "ProblemaVF",
            "nombre": getattr(problema, '_nombre', ''),
            "enunciado": problema.e,
            "NumPreguntas": problema.NumPreguntas,
            "cuestiones": [{"texto": t, "respuesta": r} for t, r in problema.c],
        }
    else:
        raise ValueError(f"Tipo no soportado: {type(problema)}")

```

Capítulo 14

Lectura y escritura de ficheros

Las funciones de alto nivel abren y cierran los ficheros JSON y delegan en las funciones de conversión.

`load_problema` detecta si el fichero contiene un banco (clave "banco") y en ese caso devuelve un error indicando al usuario que debe usar `load_banco`. Esto evita silenciosamente cargar solo el primer problema cuando el usuario ha pasado un banco por error.

Cargar y guardar problema JSON

```
def load_problema(filepath):
    """Carga un único problema desde un fichero JSON."""
    with open(filepath, 'r', encoding='utf-8') as f:
        d = _json.load(f)
    if "banco" in d:
        raise ValueError(f"{filepath!r} contiene un banco de problemas. Usa load_banco().")
    return problema_from_dict(d)

def save_problema(problema, filepath):
    """Guarda un único problema en un fichero JSON."""
    d = problema_to_dict(problema)
    with open(filepath, 'w', encoding='utf-8') as f:
        _json.dump(d, f, ensure_ascii=False, indent=2)
```

`load_banco` acepta tanto ficheros de banco (con "banco") como ficheros de problema individual, en cuyo caso devuelve una lista de un elemento. `save_banco` acepta tanto listas como diccionarios (cuyas claves se ignoran, usándose solo los valores).

Cargar y guardar banco JSON

```
def load_banco(filepath):
    """Carga un banco de problemas desde un fichero JSON.
    Devuelve una lista de ProblemaTipo / ProblemaVF."""
    with open(filepath, 'r', encoding='utf-8') as f:
        d = _json.load(f)
    if "banco" in d:
        return [problema_from_dict(p) for p in d["banco"]]
    else:
        return [problema_from_dict(d)]

def save_banco(problemas, filepath):
    """Guarda una lista (o dict) de problemas en un fichero JSON de banco."""
    items = list(problemas.values() if isinstance(problemas, dict) else problemas)
    banco = [problema_to_dict(p) for p in items]
    d = {"version": "1", "banco": banco}
    with open(filepath, 'w', encoding='utf-8') as f:
        _json.dump(d, f, ensure_ascii=False, indent=2)
```

Capítulo 15

Ejemplo de uso

El siguiente ejemplo crea un `ProblemaTipo` desde un dict, lo itera, lo guarda en disco y lo recarga comprobando que el ciclo completo de serialización es transparente:

```
from qbank import problema_from_dict, save_problema, load_problema

d = {
    "version": "1",
    "tipo": "ProblemaTipo",
    "nombre": "ejemplo_json",
    "setup": None,
    "componentes": [
        "Considere ",
        [
            {"tipo": "Supuesto", "enunciado": "A ", "semantica": "v('A')", "precond": "True"},
            {"tipo": "Supuesto", "enunciado": "B ", "semantica": "v('B')", "precond": "True"}
        ],
        [
            {"tipo": "Cuestion", "enunciado": "¿A?", "semantica": "v('A')", "precond": "True", "exp": ""},
            {"tipo": "Cuestion", "enunciado": "¿B?", "semantica": "v('B')", "precond": "True", "exp": ""}
        ]
    ]
}

p = problema_from_dict(d)
for etiqueta, enunciado, cuestiones in p:
    print(etiqueta, enunciado, cuestiones)
```

```
save_problema(p, "./tmp/ejemplo.json")
p2 = load_problema("./tmp/ejemplo.json")
for etiqueta, enunciado, cuestiones in p2:
    print(etiqueta, enunciado, cuestiones)
```

Capítulo 16

Exportación a código Python editable

Además de guardar en JSON, es posible convertir un problema a código Python puro: una lista de listas con `Supuesto` y `Cuestion` tal y como se escribiría a mano. El resultado es un fichero `.py` legible y editable con cualquier editor de texto.

16.1. `_slot_to_python`

Convierte un slot (cadena, `Supuesto`, `Cuestion` o lista de alternativas) al fragmento de código Python equivalente. El parámetro `nivel` controla la indentación (cada nivel añade cuatro espacios).

- Una cadena de texto se escribe con `_py_text`: una *raw string* (`r'...'`) cuando es seguro, para que el $\text{\LaTeX}$ se lea sin barras invertidas dobladas, o `repr()` en caso contrario.
- Un `Supuesto` o `Cuestion` (representado como dict JSON interno) se escribe como llamada al constructor con *todos* sus argumentos nombrados (`enunciado`, `semantica`, `precond` y, en `Cuestion`, `exp`), cada uno en su propia línea e indentado, incluyendo los valores por defecto. Así el guión resultante es fácil de leer y editar a mano.
- Una lista de alternativas se escribe como lista Python anidada.

La función recibe el slot en su representación de dict JSON (producida por `_slot_to_json`), no el objeto Python original, porque `problema_to_python` invoca primero `problema_to_dict` para normalizar la representación.

```
# Exportación a código Python

def _py_text(s):
    """Literal de texto Python legible: usa una raw string cuando es seguro
    (para que el LaTeX se lea sin barras invertidas dobladas) y repr() en caso
    contrario (texto con comillas conflictivas, salto de línea o backslash final)."""
    if "\n" not in s:
        trailing_bs = len(s) - len(s.rstrip("\\"))
        if trailing_bs % 2 == 0:
            if "'" not in s:
                return f'r'{s}'
            if '"' not in s:
                return f'r"{s}"'
    return repr(s)

def _slot_to_python(slot, nivel=1):
    pad = "    " * nivel

    if isinstance(slot, str):
        return f"{pad}{_py_text(slot)},"

    elif isinstance(slot, dict):
        # Constructor con todos los argumentos nombrados, uno por línea, para
        # que el guión Python resultante sea fácil de leer y editar a mano.
        tipo = slot["tipo"]
        ipad = "    " * (nivel + 1)
```



```

        args = [
            f"{ipad}enunciado = {_py_text(slot['enunciado'])}," ,
            f"{ipad}semantica = {slot['semantica']}," , # cadena evaluable: "v('A')", "True", ...
            f"{ipad}precond = {slot.get('precond', 'True')}" ,
        ]
        if tipo == "Cuestion":
            args[-1] += ","
            args.append(f"{ipad}exp = {_py_text(slot.get('exp', ''))}")
            cuerpo = "\n".join(args)
            return f"{ipad}{tipo}(\n{cuerpo}\n{ipad})"

    elif isinstance(slot, list):
        inner = "\n".join(_slot_to_python(item, nivel + 1) for item in slot)
        return f"{ipad}[\n{inner}\n{ipad}]"

    raise ValueError(f"Tipo no convertible a Python: {type(slot)}")

def _subpregunta_to_python(sp_dict, nivel=1):
    """Convierte una sub-pregunta JSON al código SubPregunta(...) equivalente."""
    pad = " " * nivel
    intro = repr(sp_dict["intro"])
    inner = "\n".join(_slot_to_python(c, nivel + 1) for c in sp_dict["cuestiones"])
    return f"{pad}SubPregunta({intro}, [\n{inner}\n{pad}])"

def problema_to_python(problema, varname="ejercicio"):
    """Genera código Python que recrea el problema como listas editables.

    Soporta ProblemaTipo y ProblemaMultiParte.
    El fichero resultante puede abrirse con cualquier editor de texto,
    modificarse y ejecutarse directamente con Python.
    Para problemas con setup, el código del setup queda como función Python
    editable (_setup). Si el setup es un callable sin _setup_str, falla.
    """
    d = problema_to_dict(problema)
    tipo = d.get("tipo", "ProblemaTipo")

    if tipo == "ProblemaMultiParte":
        imports = "from qbank import Supuesto, Cuestion, SubPregunta, ProblemaMultiParte"
    else:
        imports = "from qbank import Supuesto, Cuestion, ProblemaTipo"

    lines = [imports, "from calccprop import *", ""]

    setup_str = d.get("setup")
    if setup_str:
        if setup_str:
            lines += ["", "def _setup():"]
            for line in setup_str.splitlines():
                lines.append(f"    {line}")
            lines += [
                "    _locs = locals()",
                "    return {k: v for k, v in _locs.items() if not k.startswith('_')}",
                "",
            ]

    setup_arg = "", setup=_setup if setup_str else ""

    if tipo == "ProblemaMultiParte":
        lines.append(f"{varname} = [")
        for comp in d["componentes"]:
            lines.append(_slot_to_python(comp, nivel=1))
        lines.append("]")
        lines.append("")
        subvar = varname + "_subs"
        lines.append(f"{subvar} = [")
        for sp in d["subpreguntas"]:
            lines.append(_subpregunta_to_python(sp, nivel=1))
        lines.append("]")
        lines.append("")
        lines.append(f"p = ProblemaMultiParte({varname}, {subvar}{setup_arg})")
    else:
        lines.append(f"{varname} = [")
        for comp in d["componentes"]:
            lines.append(_slot_to_python(comp, nivel=1))
        lines.append("]")
        lines.append("")
        lines.append(f"p = ProblemaTipo({varname}{setup_arg})")

    lines.append("")
    return "\n".join(lines)

def save_problema_py(problema, filepath, varname="ejercicio"):
    """Guarda el problema como código Python editable en un fichero .py."""
    code = problema_to_python(problema, varname=varname)
    with open(filepath, "w", encoding="utf-8") as f:

```

16.2. `_subpregunta_to_python`, `problema_to_python` y `save_problema_py`

`_subpregunta_to_python` es el helper para `ProblemaMultiParte`: convierte un dict de sub-pregunta al código `SubPregunta(intro, [Cuestion(...), ...])`, equivalente, siguiendo el mismo patrón de indentación que `_slot_to_python` para listas.

`problema_to_python` es el punto de entrada público. Invoca `problema_to_dict` para obtener la representación JSON normalizada y luego genera el código Python línea a línea. El resultado es un fichero Python completo que:

1. Importa las clases correctas según el tipo (`ProblemaTipo` o `SubPregunta`, `ProblemaMultiParte`) y todo de `calcprop`.
2. Define una función `_setup()` si el problema tiene setup (el cuerpo de la función es exactamente el código Python almacenado en el campo "setup" del JSON).
3. Para `ProblemaTipo`: define la variable (`varname`) como lista de listas y construye `ProblemaTipo(varname)`.
4. Para `ProblemaMultiParte`: define (`varname`) (componentes) y (`varname`)_subs (sub-preguntas) como listas separadas y construye `ProblemaMultiParte(varname, varname_subs)`.
5. Construye `p = ProblemaTipo(ejercicio)` (o con `setup=_setup` si procede).

El patrón `_locs = locals()` en `_setup` es necesario porque en Python 3 una comprensión de lista tiene su propio ámbito; capturar las variables locales antes de la comprensión evita que las variables de iteración las sobrescriban.

`save_problema_py` simplemente escribe la cadena resultante en un fichero.

Limitaciones: solo funciona con `ProblemaTipo` (no con `ProblemaVF`). Si el problema tiene un setup definido como función Python (no cargado desde JSON), `problema_to_dict` falla antes de llegar aquí.

16.3. Ejemplo

```
from qbank import problema_to_python
print(problema_to_python(p))
```

Parte III

Editor visual de problemas en Jupyter

El módulo `qbank._widgets` proporciona `ProblemaTipoEditor`, un formulario interactivo basado en `ipywidgets` que permite construir, visualizar y guardar objetos `ProblemaTipo` desde un notebook de Jupyter sin escribir la estructura de listas a mano.

Capítulo 17

Dependencias e instalación

El módulo requiere el paquete `ipywidgets`. Si no está disponible, la importación falla silenciosamente: el bloque `try/except` de `qbank.__init__` hace que el resto del paquete siga funcionando con normalidad.

Para instalarlo en el entorno virtual:

```
pip install "calcprop-qbank[jupyter]"
```

En **JupyterLab 4** es necesario que la extensión `@jupyter-widgets/jupyterlab-manager` aparezca como habilitada en `jupyter labextension list`. Si los widgets muestran su representación textual (`VBox(children...=)` en lugar del formulario), el síntoma es que JupyterLab no está escaneando el directorio `share/jupyter/labextensions` del entorno virtual. La solución consiste en crear un enlace simbólico al directorio de extensiones del usuario:

```
mkdir -p ~/.local/share/jupyter/labextensions/@jupyter-widgets
ln -sfn /ruta/.venv/share/jupyter/labextensions/@jupyter-widgets/jupyterlab-manager \
    ~/.local/share/jupyter/labextensions/@jupyter-widgets/jupyterlab-manager
```

Tras reiniciar JupyterLab, `jupyter labextension list` debe mostrar la extensión como habilitada.

Estructura del módulo

qbank._widgets

```
__all__ = ['ProblemaTipoEditor']

qbank/_widgets.py

try:
    import ipywidgets as _w
    from IPython.display import display as _display, clear_output as _clear
    _HAS_WIDGETS = True
except ImportError:
    _HAS_WIDGETS = False

from qbank._quiz import ProblemaTipo, ProblemaTipoProfe
from qbank._json import problema_from_dict, problema_to_dict, load_problema, save_problema, problema_to_python

def _need_widgets():
    if not _HAS_WIDGETS:
        raise ImportError(
            "ipywidgets no está instalado. Ejecuta:\n"
            "  pip install ipywidgets")

<<Widget de alternativa>>
<<Widget de slot>>
<<Editor principal ProblemaTipoEditor>>
```

Capítulo 18

Widget de alternativa (_AltWidget)

El widget `_AltWidget` representa una única alternativa dentro de un slot. Sus campos son:

tipo dropdown con valores `Supuesto` / `Cuestion`.

e (**enunciado**) texto de la alternativa.

s (**semántica**) expresión calccprop (cadena), o `True` / `False`, o una lambda para problemas paramétricos ("lambda ns: ns['x'] >0").

p (**precondición**) expresión calccprop; por defecto `True`.

x (**explicación**) texto de feedback para Moodle; solo visible cuando el tipo es `Cuestion`.

El botón elimina la alternativa de su slot padre invocando el callback `_on_delete`. El método `_sync_exp` oculta o muestra el campo de explicación (y su etiqueta) según el tipo seleccionado. Nótese que la visibilidad debe establecerse a la cadena `'visible'` (y no a `'`), ya que `ipywidgets` valida el valor del atributo `visibility` del `Layout`.

```
# Widget de una alternativa (Supuesto o Cuestion)
Widget de alternativa

class _AltWidget:
    def __init__(self, d=None, on_delete=None):
        if d is None:
            d = {}
        self._on_delete = on_delete

        self.tipo = _w.Dropdown(
            options=['Supuesto', 'Cuestion'],
            value=d.get('tipo', 'Cuestion'),
            layout=_w.Layout(width='95px'))
        self.e = _w.Text(
            value=d.get('enunciado', ''), placeholder='enunciado',
            layout=_w.Layout(width='210px'))
        self.s = _w.Text(
            value=d.get('semantica', 'True'), placeholder="v('A')",
            layout=_w.Layout(width='130px'))
        self.p = _w.Text(
            value=d.get('precond', 'True'), placeholder='precond',
            layout=_w.Layout(width='70px'))
        self.x = _w.Text(
            value=d.get('exp', ''), placeholder='explicación',
            layout=_w.Layout(width='120px'))
        self._exp_lbl = _w.Label('exp:', layout=_w.Layout(width='30px'))

        del_btn = _w.Button(
            description='', button_style='danger',
            layout=_w.Layout(width='30px', height='28px'))
        del_btn.on_click(lambda _: self._on_delete(self) if self._on_delete else None)
        self.tipo.observe(lambda _: self._sync_exp(), names='value')
        self._sync_exp()

    self.box = _w.HBox([
        self.tipo,
        _w.Label('e:', layout=_w.Layout(width='15px')), self.e,
        _w.Label('s:', layout=_w.Layout(width='15px')), self.s,
        _w.Label('p:', layout=_w.Layout(width='15px')), self.p,
        self._exp_lbl, self.x,
        del_btn,
```

```
    })

    def _sync_exp(self):
        vis = 'visible' if self.tipo.value == 'Cuestion' else 'hidden'
        self.x.layout.visibility = vis
        self._exp_lbl.layout.visibility = vis

    def to_dict(self):
        d = {'tipo': self.tipo.value, 'enunciado': self.e.value,
            'semantica': self.s.value, 'precond': self.p.value}
        if self.tipo.value == 'Cuestion':
            d['exp'] = self.x.value
        return d
```

Capítulo 19

Widget de slot (`_SlotWidget`)

Un slot es uno de los «huecos» estructurales del problema. Puede ser:

- **texto:** texto fijo que aparece igual en todas las variantes (una cadena Python).
- **alternativas:** una o varias filas `_AltWidget` de las que `ProblemaTipo` elige una por variante.

El dropdown `_tipo_dd` permite cambiar el modo. En modo `alternativas`, el botón `+ alt` añade una nueva fila vacía; el método `_del_alt` (conectado al botón `-` de cada `_AltWidget`) la elimina. El botón del slot elimina el slot completo llamando al callback `_on_delete` del editor padre.

El método `update_label` actualiza el número de slot tras añadir o eliminar slots anteriores. El método `to_json` devuelve el slot en el formato esperado por `problema_from_dict`: una cadena si es texto, o una lista de dicts si es alternativas.

```
# Widget de un slot (texto / lista de alternativas) Widget de slot

class _SlotWidget:
    def __init__(self, data=None, on_delete=None, index=0):
        self._on_delete = on_delete
        self._alts = []

        if data is None or isinstance(data, str):
            tipo_init, text_init, alts_init = 'texto', data or '', []
        elif isinstance(data, list) and len(data) == 1 and isinstance(data[0], str):
            # estructura antigua (lista de listas): un texto envuelto en lista
            # [ 'texto' ] se interpreta como slot de texto, no de alternativas.
            tipo_init, text_init, alts_init = 'texto', data[0], []
        elif isinstance(data, dict):
            # single component
            tipo_init, text_init, alts_init = 'alternativas', '', [data]
        else:
            # list of components
            tipo_init, text_init, alts_init = 'alternativas', '', data

        self._tipo_dd = _w.Dropdown(
            options=['texto', 'alternativas'], value=tipo_init,
            layout=_w.Layout(width='110px'))
        self._lbl = _w.Label(
            f'Slot {index + 1}', layout=_w.Layout(width='52px'))

        self._text = _w.Text(
            value=text_init, placeholder='texto del enunciado...',
            layout=_w.Layout(width='360px'))

        self._alts_box = _w.VBox([])
        _add_btn = _w.Button(
            description='+ alt', button_style='info',
            layout=_w.Layout(width='70px', height='26px'))
        _add_btn.on_click(lambda _: self._add_alt())
        self._alts_area = _w.VBox([self._alts_box, _add_btn])

        del_btn = _w.Button(
            description='-', button_style='danger',
            layout=_w.Layout(width='30px', height='28px'))
        del_btn.on_click(lambda _: self._on_delete(self) if self._on_delete else None)

        self._content = _w.HBox([self._text])
        self._tipo_dd.observe(self._on_tipo, names='value')
        self._box = _w.VBox([
            _w.HBox([self._lbl, self._tipo_dd, self._content, del_btn])
        ])
```

```

        if tipo_init == 'alternativas':
            self._content.children = [self._alts_area]
            for a in alts_init:
                self._add_alt(a)

    def _on_tipo(self, change):
        self._content.children = (
            [self._text] if change['new'] == 'texto' else [self._alts_area])

    def _add_alt(self, d=None):
        alt = _AltWidget(d=d, on_delete=self._del_alt)
        self._alts.append(alt)
        self._alts_box.children = [a.box for a in self._alts]

    def _del_alt(self, alt):
        self._alts = [a for a in self._alts if a is not alt]
        self._alts_box.children = [a.box for a in self._alts]

    def update_label(self, i):
        self._lbl.value = f'Slot {i + 1}'

    def to_json(self):
        if self._tipo_dd.value == 'texto':
            return self._text.value
        return [a.to_dict() for a in self._alts]

```

Capítulo 20

Editor principal (ProblemaTipoEditor)

`ProblemaTipoEditor` es el widget de nivel superior. Al instanciarse llama a `_display(self._ui)`, lo que hace que el formulario se renderice inmediatamente en la celda del notebook.

El parámetro `source` admite cuatro tipos:

None el editor comienza vacío.

Cadena de texto ruta a un fichero JSON; el fichero se carga con `load_problema` y el campo `filepath` se actualiza automáticamente con la ruta indicada.

dict se interpreta directamente como descripción del problema (en el mismo formato JSON).

ProblemaTipo el objeto (que debe llevar el atributo `_json` en sus componentes, es decir, debe haber sido cargado con `load_problema` o `problema_from_dict`) se serializa con `problema_to_dict` y se muestra en el editor.

Los botones de la barra inferior:

Botón	Acción
Preview	Vista «profe»: para cada combinación de supuestos muestra hasta N variantes con todas sus cuestiones
Guardar	Serializa el estado actual y lo guarda en el fichero indicado en el campo <code>filepath</code> .
Cargar	Lee el fichero indicado y actualiza el editor con su contenido.
\{ \} JSON	Muestra el JSON equivalente al estado actual del editor en el área de salida.
.json	Muestra en el área de salida un enlace HTML (<code>data URI</code> , atributo <code>download</code>) para guardar el JSON en

El método `to_dict()` devuelve el dict JSON del estado actual del editor. El método `to_problema()` devuelve un `ProblemaTipo` listo para iterar, construyéndolo con `problema_from_dict(self.to_dict())`.

```
# Editor principal
class ProblemaTipoEditor:
    """Editor visual de ProblemaTipo para Jupyter.

    Parameters
    -----
    source : str | dict | ProblemaTipo | None
        Fuente inicial. Puede ser la ruta a un fichero JSON, un dict,
        un objeto ProblemaTipo (creado con load_problema), o None para
        empezar un problema nuevo.
    """

    def __init__(self, source=None):
        _need_widgets()
        self._slots = []

        # Cabecera
        self._nombre = _w.Text(
            value='', placeholder='nombre del ejercicio',
            layout=_w.Layout(width='320px'))
        self._setup = _w.Textarea()
```

```

        value='', placeholder='código Python del setup (opcional)',
        layout=_w.Layout(width='520px', height='58px'))

header = _w.VBox([
    _w.HBox([_w.Label('Nombre:', layout=_w.Layout(width='65px')),
        self._nombre]),
    _w.HBox([_w.Label('Setup:', layout=_w.Layout(width='65px')),
        self._setup]),
])

# Zona de slots
self._slots_box = _w.VBox([])
add_slot_btn = _w.Button(
    description='+ slot', button_style='success',
    layout=_w.Layout(width='85px'))
add_slot_btn.on_click(lambda _: self._add_slot())

# Controles inferiores
self._filepath = _w.Text(
    value='problema.json', placeholder='ruta del fichero JSON',
    layout=_w.Layout(width='260px'))
self._n_prev = _w.BoundedIntText(
    value=5, min=1, max=100,
    layout=_w.Layout(width='55px'))

prev_btn = _w.Button(description=' Preview', button_style='primary',
    layout=_w.Layout(width='105px'))
save_btn = _w.Button(description=' Guardar', layout=_w.Layout(width='95px'))
load_btn = _w.Button(description=' Cargar', layout=_w.Layout(width='95px'))
json_btn = _w.Button(description='{ } JSON', layout=_w.Layout(width='95px'))
dl_btn = _w.Button(description='.json', layout=_w.Layout(width='85px'))
py_btn = _w.Button(description='.py', layout=_w.Layout(width='80px'))

prev_btn.on_click(self._on_preview)
save_btn.on_click(self._on_save)
load_btn.on_click(self._on_load)
json_btn.on_click(self._on_show_json)
dl_btn.on_click(self._on_download)
py_btn.on_click(self._on_download_py)

self._out = _w.Output()

self._ui = _w.VBox([
    header,
    _w.HTML('<hr style="margin:4px 0">'),
    self._slots_box,
    add_slot_btn,
    _w.HTML('<hr style="margin:4px 0">'),
    _w.HBox([
        prev_btn,
        _w.Label('vars:', layout=_w.Layout(width='32px')),
        self._n_prev,
        _w.Label(' '),
        save_btn, load_btn,
        _w.Label(' '),
        self._filepath,
        _w.Label(' '),
        json_btn, dl_btn, py_btn,
    ]),
    self._out,
])

# Carga inicial
if source is not None:
    self._load_source(source)

_display(self._ui)

# Gestión de slots

def _add_slot(self, data=None):
    s = _SlotWidget(data=data, on_delete=self._del_slot,
        index=len(self._slots))
    self._slots.append(s)
    self._refresh()

def _del_slot(self, slot):
    self._slots = [s for s in self._slots if s is not slot]
    self._refresh()

def _refresh(self):
    for i, s in enumerate(self._slots):
        s.update_label(i)
    self._slots_box.children = [s.box for s in self._slots]

# Carga de datos

```

```

def _load_source(self, source):
    if isinstance(source, str):
        p = load_problema(source)
        self._filepath.value = source
        d = problema_to_dict(p)
    elif isinstance(source, ProblemaTipo):
        d = problema_to_dict(source)
    elif isinstance(source, dict):
        d = source
    else:
        raise ValueError(f"Fuente no reconocida: {type(source)}")

    self._nombre.value = d.get('nombre', '')
    self._setup.value = d.get('setup', '') or ''
    self._slots = []
    for comp in d.get('componentes', []):
        self._add_slot(data=comp)

# Serialización

def to_dict(self):
    """Devuelve el problema actual como dict JSON."""
    setup = self._setup.value.strip() or None
    return {
        'version': '1',
        'tipo': 'ProblemaTipo',
        'nombre': self._nombre.value,
        'setup': setup,
        'componentes': [s.to_json() for s in self._slots],
    }

def to_problema(self):
    """Devuelve un ProblemaTipo listo para iterar."""
    return problema_from_dict(self.to_dict())

# Callbacks de botones

def _on_preview(self, _):
    with self._out:
        _clear()
        try:
            # Vista «profe»: para cada combinación de supuestos se muestran
            # todas las cuestiones posibles (sin descartar las inconsistentes),
            # marcando cada una como correcta/incorrecta.
            p = self.to_problema()
            n = self._n_prev.value
            cnt = 0
            for var in ProblemaTipoProfe(p.e, setup=p.setup):
                if cnt >= n:
                    break
                id_, enunciado, cuestiones = var
                print(f" Variante {id_} {enunciado}")
                for c in cuestiones:
                    mark = ' ' if c[1] is True else (' ' if c[1] is False else '?')
                    print(f" {mark} {c[0]}")
                cnt += 1
            if cnt == 0:
                print("(sin variantes)")
        except Exception as exc:
            print(f"Error: {exc}")

def _on_save(self, _):
    with self._out:
        _clear()
        try:
            path = self._filepath.value.strip()
            p = problema_from_dict(self.to_dict())
            save_problema(p, path)
            print(f" Guardado en {path!r}")
        except Exception as exc:
            print(f"Error al guardar: {exc}")

def _on_load(self, _):
    with self._out:
        _clear()
        try:
            path = self._filepath.value.strip()
            self._slots = []
            self._load_source(path)
            print(f" Cargado desde {path!r}")
        except Exception as exc:
            print(f"Error al cargar: {exc}")

def _on_show_json(self, _):
    with self._out:
        _clear()
        try:

```

```

import json
print(json.dumps(self.to_dict(), ensure_ascii=False, indent=2))
except Exception as exc:
    print(f"Error: {exc}")

def _on_download(self, _):
    # JupyterLab no ejecuta el JS de IPython.display.Javascript desde un
    # callback de botón (solo muestra «<IPython.core.display.Javascript object>»).
    # En su lugar mostramos un enlace HTML con el JSON embebido como data URI
    # (atributo `download`): el navegador lo descarga al pulsarlo, sin ejecutar JS.
    import json as _j
    import os, base64
    from IPython.display import HTML
    with self._out:
        _clear()
        try:
            data = _j.dumps(self.to_dict(), ensure_ascii=False, indent=2)
            filename = os.path.basename(self._filepath.value.strip() or 'problema.json')
            b64 = base64.b64encode(data.encode('utf-8')).decode('ascii')
            href = f"data:application/json;base64,{b64}"
            _display(HTML(
                f'<a download="{filename}" href="{href}" '
                f'style="font-size:14px;font-weight:bold">'
                f' Descargar {filename}</a>'
                f'<br><span style="color:gray">(pulsar el enlace para guardar el fichero)</span>'))
        except Exception as exc:
            print(f"Error al descargar: {exc}")

def _on_download_py(self, _):
    # Igual que _on_download pero exporta el problema como guión Python
    # editable (problema_to_python). El nombre del fichero se deriva de la
    # ruta JSON cambiando la extensión a .py.
    import os, base64
    from IPython.display import HTML
    with self._out:
        _clear()
        try:
            p = problema_from_dict(self.to_dict())
            code = problema_to_python(p)
            base = os.path.basename(self._filepath.value.strip() or 'problema.json')
            filename = os.path.splitext(base)[0] + '.py'
            b64 = base64.b64encode(code.encode('utf-8')).decode('ascii')
            href = f"data:text/x-python;base64,{b64}"
            _display(HTML(
                f'<a download="{filename}" href="{href}" '
                f'style="font-size:14px;font-weight:bold">'
                f' Descargar {filename}</a>'
                f'<br><span style="color:gray">(pulsar el enlace para guardar el fichero)</span>'))
        except Exception as exc:
            print(f"Error al descargar: {exc}")

```

Parte IV

¿Y ahora qué?

20.1. ¿Qué tenemos?

- Contamos con un módulo de cálculo proposicional que determina la veracidad o falsedad de cada enunciado, basada en los supuestos de cada caso particular.
- Poseemos una metodología para programar ejercicios utilizando listas de sublistas que incluyen textos, supuestos y cuestiones.
- Tenemos la capacidad de desempaquetar dichas listas mediante **ProblemaTipo**, lo que nos permite generar múltiples ejercicios, con la tranquilidad de que el módulo de cálculo proposicional se encarga de identificar qué opciones son verdaderas y cuáles son falsas, de acuerdo con la composición de los supuestos de cada enunciado.

Con esto, hemos desarrollado un procedimiento para generar un amplio conjunto de variantes de ejercicios de opción múltiple.

20.2. ¿Qué nos falta?

Falta dar formato a lo que hemos desarrollado para ser usado con los estudiantes. El módulo **CalcProp-export** nos permitirá crear bancos de preguntas en formato $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$, que podrán ser utilizados con [Auto Multiple Choice \(AMC\)](#), así como también generar bancos en formato `xml` para su empleo en el entorno Moodle mediante el paquete de $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ [moodle](#), específicamente para cuestiones de [opción múltiple](#).